Contents

Contents 1

Appendix A Computations in GAP 1
A.1 5-cell . . . 1
A.2 16-cell . . . 2
A.2.1 Cycles of the 16-cell . . . 2
A.2.2 Candidates for S_2 in the 16-cell . . . 4
A.2.3 Structures of type $\mathsf{P}(S_1, S_2; v)$ with skew faces that are not polyhedra . . . 4
A.2.4 Structures of type $\mathsf{P}(S_1, S_2; v)$ with helical faces that are not polyhedra . . . 6
A.2.5 A regular polyhedron with helical faces in the 16-cell . . . 7
A.3 8-cell and 24-cell . . . 7
A.3.1 Cycles of the 24-cell . . . 7
A.3.2 Structures of type $\mathsf{P}(S_1, S_2; v)$ with skew faces that are not polyhedra . . . 9
A.3.3 Structures of type $\mathsf{P}(S_1, S_2; v)$ with helical faces that are not polyhedra . . . 10
A.4 120-cell . . . 11
A.4.1 Cycles of the 120-cell . . . 11
A.4.2 Candidates for S_2 in the 120-cell for skew S_1 . . . 14
A.4.3 Structures of type $\mathsf{P}(S_1, S_2; v)$ with skew faces that are not polyhedra . . . 15
A.4.4 Candidates for S_2 in the 120-cell for helical S_1 . . . 15
A.5 600-cell . . . 18
A.5.1 Cycles of the 600-cell . . . 18
A.5.2 Candidates for S_2 in the 600-cell for skew S_1 . . . 24
A.5.3 Structures of type $\mathsf{P}(S_1, S_2; v)$ with skew faces that are not polyhedra . . . 27
A.5.4 Candidates for S_2 in the 600-cell for helical S_1 . . . 30
A.5.5 Structures of type $\mathsf{P}(S_1, S_2; v)$ with helical faces that are not polyhedra . . . 34
A.5.6 A chiral polyhedron with helical faces, derived from the 600-cell . . . 36
A.6 Chiral 4-polytopes . . . 37
A.6.1 Searching for S_3 in the 5-cell . . . 41
A.6.2 Searching for S_3 in the 5-simplex . . . 43
A.6.3 Searching for S_3 in the 8-cell . . . 45
A.6.4 Searching for S_3 in the 24-cell . . . 47
A.6.5 Searching for S_3 in the 120-cell . . . 49
A.6.6 Searching for S_3 in the 600-cell . . . 50

Bibliography 53

APPENDIX A

Computations in GAP

The following computations were carried out in GAP ([4]).

A.1 5-cell

Distinguished generators of the 5-cell and base vertex $v = 1$:

```
gap> r0:=(1,2);
(1,2)
gap> r1:=(2,3);
(2,3)
gap> r2:=(3,4);
(3,4)
gap> r3:=(4,5);
(4,5)
gap> v:=1;
1
```

Define $S_1 := R_0R_1R_2R_3$.

```
gap> S1:=r0*r1*r2*r3;
(1,5,4,3,2)
```

Define S_2 by $S_2 := R_3R_2R_1R_2$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r3*r2*r1*r2;
(2,4,5)
gap> G:=Group(S1,S2);
Group([ (1,5,4,3,2), (2,4,5) ])
gap> Size(Orbit(G, v, OnPoints));
5
gap> H:=Group(S2);;
gap> Index(G, H);
20
```

Define S_2 by $S_2 := R_3R_2R_1R_3$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r3*r2*r1*r3;
(2,3,5)
```

```
gap> G:=Group(S1,S2);
Group([ (1,5,4,3,2), (2,3,5) ])
gap> Size(Orbit(G, v, OnPoints));
5
gap> H:=Group(S2);;
gap> Index(G, H);
20
```

A.2 16-cell

A.2.1 Cycles of the 16-cell

Distinguished generators of the 16-cell:

```
gap> r0:=(1,2)(5,6);
(1,2)(5,6)
gap> r1:=(2,3)(6,7);
(2,3)(6,7)
gap> r2:=(3,4)(7,8);
(3,4)(7,8)
gap> r3:=(4,8);
(4,8)
gap> P:=Group(r0,r1,r2,r3);
Group([ (1,2)(5,6), (2,3)(6,7), (3,4)(7,8), (4,8) ])
gap> eGC:=Elements(P);;
```

We discard involutions and symmetries of order three (since these correspond to the triangular faces of the 16-cell, and these cycles lie in a plane).

```
gap> filP:=[];; ##Filtro de P.
gap> for j in [1..Length(eGC)] do
>       if Order(eGC[j])=2 or Order(eGC[j])=3 then
>               continue;
>       else
>               Add(filP,eGC[j]);
>       fi;
> od;
gap> filP;;
gap> Length(filP);
277
gap> #Since the base edge is {1, 2} and the base vertex is 1,
#we want a rotation that maps 1 to 2 upon the
#first rotation.
gap> pHel:=[];;
gap> for i in [1..Length(filP)] do
>       if Length(Cycle(filP[i],1))>1 and Cycle(filP[i],1)[2]=2 then
>               Add(pHel,filP[i]);
>       fi;
> od;
gap> pHel;
[ (1,2)(3,4,7,8)(5,6), (1,2)(3,8,7,4)(5,6), (1,2,3)(4,8)(5,6,7),
  (1,2,3,4)(5,6,7,8), (1,2,3,4,5,6,7,8), (1,2,3,5,6,7),
  (1,2,3,5,6,7)(4,8), (1,2,3,8,5,6,7,4), (1,2,3,8)(4,5,6,7),
  (1,2,4,3)(5,6,8,7), (1,2,4,7,5,6,8,3), (1,2,4,5,6,8),
  (1,2,4,3,5,6,8,7), (1,2,4,7)(3,5,6,8), (1,2,4)(3,7)(5,6,8),
```

```
  (1,2,4,5,6,8)(3,7), (1,2,5,6), (1,2,5,6)(4,8), (1,2,5,6)(3,4)(7,8),
  (1,2,5,6)(3,4,7,8), (1,2,5,6)(3,7), (1,2,5,6)(3,7)(4,8),
  (1,2,5,6)(3,8,7,4), (1,2,5,6)(3,8)(4,7), (1,2,7,5,6,3),
  (1,2,7,5,6,3)(4,8), (1,2,7,8,5,6,3,4), (1,2,7,8)(3,4,5,6),
  (1,2,7)(3,5,6)(4,8), (1,2,7,4)(3,8,5,6), (1,2,7,4,5,6,3,8),
  (1,2,8,7,5,6,4,3), (1,2,8,3)(4,7,5,6), (1,2,8,5,6,4),
  (1,2,8,7)(3,5,6,4), (1,2,8,3,5,6,4,7), (1,2,8,5,6,4)(3,7),
  (1,2,8)(3,7)(4,5,6) ]
gap> Length(pHel);
38
gap> #Obtaining the orders of each symmetry in PHel.
gap>  PHel:=pHel;;
gap> OrdH:=[];;
gap> for j in [1..Length(pHel)] do
>       OrdH[j]:=Order(pHel[j]);
> od;
gap> OrdH;
[ 4, 4, 6, 4, 8, 6, 6, 8, 4, 4, 8, 6, 8, 4, 6, 6, 4, 4, 4, 4, 4, 4,
 4, 4, 6, 6, 8, 4, 6, 4, 8, 8, 4, 6, 4, 8, 6, 6 ]
#Next, we will illustrate the symmetries of order 4 that are
#conjugate; the programming for the symmetries of the
#remaining orders is analogous.
#First, we will group the symmetries of order 4.
gap> SimOrd4:=[];; #Cycles (S1) of order 4.
gap> for j in [1..Length(pHel)] do
>       if Order(pHel[j])=4 then
>               Add(SimOrd4,pHel[j]);
>       fi;
> od;
gap> SimOrd4;
[ (1,2)(3,4,7,8)(5,6), (1,2)(3,8,7,4)(5,6), (1,2,3,4)(5,6,7,8),
  (1,2,3,8)(4,5,6,7), (1,2,4,3)(5,6,8,7), (1,2,4,7)(3,5,6,8),
  (1,2,5,6), (1,2,5,6)(4,8), (1,2,5,6)(3,4)(7,8), (1,2,5,6)(3,4,7,8),
  (1,2,5,6)(3,7), (1,2,5,6)(3,7)(4,8), (1,2,5,6)(3,8,7,4),
  (1,2,5,6)(3,8)(4,7), (1,2,7,8)(3,4,5,6), (1,2,7,4)(3,8,5,6),
  (1,2,8,3)(4,7,5,6), (1,2,8,7)(3,5,6,4) ]
gap> Length(SimOrd4);
18
#Symmetries of order 4 are conjugate.
gap> ElConj4:=[];;
gap>    for j in [2..Length(SimOrd4)] do
>               for k in [1..Length(eGC)] do
>                       if  eGC[k]*SimOrd4[1]*eGC[k]^-1= SimOrd4[j]
then
>                               Add(ElConj4,
[j,Factorization(P,SimOrd4[j]),Factorization(P,eGC[k])]);
>                       else
>                               continue;
>                       fi;
>               od;
>       od;
gap> ElConj4;; #Conjugate symmetries are stored here.
```

A.2.2 Candidates for S_2 in the 16-cell

ElFigV returns the symmetry group of the vertex figure of the 16-cell. These symmetries fix the base vertex $v = 1$.

The goal is find polyhedra with skew faces, so we search within the group of symmetries of the vertex figure of 16-cell, the candidate symmetries to construct a skew vertex figure are shown in red, because these are all the symmetries that are not involutions and send the 4 to 2 and those that do not construct skew cycles are discarded ((2,3,4)(6,7,8), (2,7,4)(3,8,6), (2,8,6,4), (2,8,6,4)(3,7)).

On the other hand, in the search for polyhedra with helical faces, all the symmetries that are not involutions and that send 8 to 2 are marked in blue. As we are looking to construct planar vertex figures, the permutations (2,3,4,6,7,8) and (2,7,4,6,3,8) are discarded since they form skew cycles.

```
gap> GFigVer:=Group(r1,r2,r3);
Group([ (2,3)(6,7), (3,4)(7,8), (4,8) ])
gap> ElFigV:=Elements(GFigVer);
```

[(), (4,8), (3,4)(7,8), (3,4,7,8), (3,7), (3,7)(4,8), (3,8,7,4), (3,8)(4,7), (2,3)(6,7), (2,3)(4,8)(6,7), (2,3,4)(6,7,8), (2,3,4,6,7,8), (2,3,6,7), (2,3,6,7)(4,8), (2,3,8,6,7,4), (2,3,8)(4,6,7), (2,4,3)(6,8,7), (2,4,7,6,8,3), (2,4)(6,8), (2,4,6,8), (2,4,3,6,8,7), (2,4,7)(3,6,8), (2,4)(3,7)(6,8), (2,4,6,8)(3,7), (2,6), (2,6)(4,8), (2,6)(3,4)(7,8), (2,6)(3,4,7,8), (2,6)(3,7), (2,6)(3,7)(4,8), (2,6)(3,8,7,4), (2,6)(3,8)(4,7), (2,7,6,3), (2,7,6,3)(4,8), (2,7,8,6,3,4), (2,7,8)(3,4,6), (2,7)(3,6), (2,7)(3,6)(4,8), (2,7,4)(3,8,6), (2,7,4,6,3,8), (2,8,7,6,4,3), (2,8,3)(4,7,6), (2,8,6,4), (2,8)(4,6), (2,8,7)(3,6,4), (2,8,3,6,4,7), (2,8,6,4)(3,7), (2,8)(3,7)(4,6)]

```
gap> Length(ElFigV);
48
```

A.2.3 Structures of type $\mathsf{P}(S_1, S_2; v)$ with skew faces that are not polyhedra

The base vertex of the 16-cell is $v = 1$:

```
gap> v:=1;
1
```

Define $S_1 := R_0R_1R_2$.

```
gap> S1:=r0*r1*r2;
(1,4,3,2)(5,8,7,6)
```

Define S_2 by $S_2 := R_2R_3R_1$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r2*r3*r1;
(2,3,8,6,7,4)
gap> Order(S1*S2);
2
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
gap> G:=Group(S1,S2);
Group([ (1,4,3,2)(5,8,7,6), (2,3,8,6,7,4) ])
gap> Size(Orbit(G, v, OnPoints));
8
gap> H:=Group(S2);;
gap> Index(G, H);
64
```

Define S_2 by $S_2 := R_2R_1R_2R_3R_2$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r2*r1*r2*r3*r2;
(2,7,8,6,3,4)
gap> Order(S1*S2);
2
```

```
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
gap> G:=Group(S1,S2);
Group([ (1,4,3,2)(5,8,7,6), (2,7,8,6,3,4) ])
gap> Size(Orbit(G, v, OnPoints));
8
gap> H:=Group(S2);;
gap> Index(G, H);
64
```

Define $S_1 := R_0R_1R_3R_2R_1$.

```
gap> S1:=r0*r1*r3*r2*r1;
(1,4,6,5,8,2)
```

Define S_2 by $S_2 := R_2R_3R_1$ and compute $|vG|$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r2*r3*r1;
(2,3,8,6,7,4)
gap> Order(S1*S2);
2
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
gap> G:=Group(S1,S2);
Group([ (1,4,6,5,8,2), (2,3,8,6,7,4) ])
gap> Size(Orbit(G, v, OnPoints));
8
gap> H:=Group(S2);;
gap> Index(G, H);
32
```

Define S_2 by $S_2 := R_2R_1R_2R_3R_2$ and compute $|vG|$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r2*r1*r2*r3*r2;
(2,7,8,6,3,4)
gap> Order(S1*S2);
2
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
gap> G:=Group(S1,S2);
Group([ (1,4,6,5,8,2), (2,7,8,6,3,4) ])
gap> Size(Orbit(G, v, OnPoints));
8
gap> H:=Group(S2);;
gap> Index(G, H);
32
```

Define $S_1 := R_0R_3R_2R_1R_3R_2$.

```
gap> S1:=r0*r3*r2*r1*r3*r2;
(1,4,6,5,8,2)(3,7)
```

Define S_2 by $S_2 := R_2R_3R_1$ and verify that $(S_1S_2)^2 \neq 1$.

```
gap> S2:=r2*r3*r1;
(2,3,8,6,7,4)
gap> Order(S1*S2);
4
```

Define S_2 by $S_2 := R_2R_1R_2R_3R_2$ and verify that $(S_1S_2)^2 \neq 1$.

```
gap> S2:=r2*r1*r2*r3*r2;
(2,7,8,6,3,4)
gap> Order(S1*S2);
4
```

A.2.4 Structures of type $\mathsf{P}(S_1, S_2; v)$ with helical faces that are not polyhedra

The base vertex of the 16-cell is $v = 1$:

```
gap> v:=1;
1
```

Define $S_1 := R_0R_1R_2R_3$.

```
gap> S1:=r0*r1*r2*r3;
(1,8,7,6,5,4,3,2)
```

Define S_2 by $S_2 := R_3R_2R_1R_3$ and compute $|vG|$ and the index $[G, \langle S_2 \rangle]$.

```
gap> S2:=r3*r2*r1*r3;
(2,3,8)(4,6,7)
gap> Order(S1*S2);
2
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
gap> G:=Group(S1,S2);
Group([ (1,8,7,6,5,4,3,2), (2,3,8)(4,6,7) ])
gap> Size(Orbit(G, v, OnPoints));
8
gap> H:=Group(S2);;
gap> Index(G, H);
16
```

Define S_2 by $S_2 := (R_1R_3R_2)^2$ and compute $|vG|$ and the index $[G, \langle S_2 \rangle]$.

```
gap> S2:=(r1*r3*r2)^2;
(2,7,8)(3,4,6)
gap> Order(S1*S2);
2
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
gap> G:=Group(S1,S2);
Group([ (1,8,7,6,5,4,3,2), (2,7,8)(3,4,6) ])
gap> Size(Orbit(G, v, OnPoints));
8
gap> H:=Group(S2);;
gap> Index(G, H);
16
```

Define S_2 by $S_2 := R_3R_2R_3R_1R_2$ and verify that $(S_1S_2)^2 \neq 1$.

```
gap> S2:=r3*r2*r3*r1*r2;
(2,4,6,8)(3,7)
gap> Order(S1*S2);
4
```

A.2.5 A regular polyhedron with helical faces in the 16-cell

We verify that the hypotheses of Theorem that guarantees that the structure $\mathsf{P}(S_1, S_2; v)$ is a rotary polyhedron and we get a polyhedron of type $\{8, 4\}$.

```
gap> s1:=r0*r1*r2*r3;
(1,8,7,6,5,4,3,2)
gap> s2:=r3*r2*r1*r2;
(2,4,6,8)
```

Order of S_1S_2 and the intersection property:

```
gap> Order(S1*S2);
2
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
```

Compute the right transversals of $\langle S_1\rangle$, $\langle S_1S_2\rangle$, and $\langle S_2\rangle$ in $G = \langle S_1, S_2\rangle$.

```
gap> G:=Group(s1,s2);
Group([ (1,8,7,6,5,4,3,2), (2,4,6,8) ])
gap> Size(G);
32
gap> genS1:=Group(s1);
Group([ (1,8,7,6,5,4,3,2) ])
gap> Elements(RightTransversal(G,genS1));
[ (), (2,4,6,8), (2,6)(4,8), (2,8,6,4) ]
gap> genS1S2:=Group(s1*s2);
Group([ (1,2)(3,4)(5,6)(7,8) ])
gap> Elements(RightTransversal(G,genS1S2));
[ (), (2,4,6,8), (2,6)(4,8), (2,8,6,4), (1,2,3,4,5,6,7,8),
  (1,2,5,6)(3,4,7,8), (1,2,7,8,5,6,3,4), (1,3,5,7)(2,4,6,8),
  (1,3,5,7)(2,6)(4,8), (1,3,5,7)(2,8,6,4), (1,4,7,2,5,8,3,6),
  (1,4)(2,7)(3,6)(5,8), (1,5)(2,6)(3,7)(4,8), (1,5)(2,8,6,4)(3,7),
  (1,6,3,8,5,2,7,4), (1,7,5,3)(2,8,6,4) ]
gap> genS2:=Group(s2);
Group([ (2,4,6,8) ])
gap> Elements(RightTransversal(G,genS2));
[ (), (1,2,3,4,5,6,7,8), (1,3,5,7)(2,4,6,8), (1,4,7,2,5,8,3,6),
  (1,5)(2,6)(3,7)(4,8), (1,6,3,8,5,2,7,4), (1,7,5,3)(2,8,6,4),
  (1,8,7,6,5,4,3,2) ]
```

A.3 8-cell and 24-cell

A.3.1 Cycles of the 24-cell

Distinguished generators of the 24-cell:

```
gap> r0:=(1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15)(19,23);
(1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15)(19,23)
gap> r1:=(2,24)(4,21)(5,19)(7,18)(8,14)(10,17)(12,20)(13,22)(15,23);
(2,24)(4,21)(5,19)(7,18)(8,14)(10,17)(12,20)(13,22)(15,23)
gap> r2:=(5,13)(6,14)(7,15)(8,16)(17,20)(21,24);
(5,13)(6,14)(7,15)(8,16)(17,20)(21,24)
gap> r3:=(2,13)(3,16)(6,9)(7,12)(18,20)(22,24);
(2,13)(3,16)(6,9)(7,12)(18,20)(22,24)
gap> P:=Group(r0,r1,r2,r3);
```

```
<permutation group with 4 generators>
gap> eGC:=Elements(P);;
gap> v:=11;
11
gap> e:=10;
10
gap> #We will remove symmetries of order 2 and 3.
gap> filP:=[];; ##Filtro de P.
gap> for j in [1..Length(eGC)] do
>       if (Order(eGC[j])=2 or Order(eGC[j])=3)=true then
>               continue;
>       else
>               Add(filP,eGC[j]);
>       fi;
> od;
gap> filP;;
gap> Length(filP);
933
gap> #The base vertex is 11, as it is a vertex that remains fixed under
#the three reflections: r1, r2, r3.
gap> #The base vertex is 11 and the base edge is {11, 10}; therefore,
#we want a rotation that maps 11 to 10 upon the first rotation.
gap> pHel:=[];; #Potential S1 cycles will be stored here.
gap> for i in [1..Length(filP)] do
>       if Length(Cycle(filP[i],v))>1 and Cycle(filP[i],v)[2]=e
                and Length(Cycle(filP[i],v))=Order(filP[i]) then
>               Add(pHel,filP[i]);
>       fi;
> od;
gap> pHel;
[ (1,4,3,2)(5,8,7,6)(9,12,11,10)(13,16,15,14)(18,19,22,23),
  (1,4,3,2)(5,16,7,14)(6,13,8,15)(9,12,11,10)(17,20)(18,19,22,23)(21,24),
  (1,4,8,7,6,2)(3,5)(9,15)(10,14,13,16,12,11)(17,18,19,21,22,23),
  (1,4,8,12,11,10,14,2)(3,5,16,7,9,15,6,13)(17,20,18,19,21,24,22,23),
  (1,4,16,12,11,10,6,2)(3,13,8,15,9,7,14,5)(17,18,19,24,21,22,23,20),
  (1,4,16,15,14,2)(3,13)(5,8,12,11,10,6)(7,9)(18,19,24,22,23,20),
  (1,4,8,5)(2,3,7,6)(9,13,16,12)(10,14,15,11)(17,19,21,23),
  (1,4,8,12,9,13)(2,3,7,11,10,14)(5,16)(6,15)(17,20,19,21,24,23),
  (1,4,16,12,9,5)(2,3,15,11,10,6)(7,14)(8,13)(17,19,24,21,23,20),
  (1,4,16,13)(2,3,15,14)(5,8,12,9)(6,7,11,10)(19,24,23,20),
  (1,4,3,7,6,5)(2,8)(9,13,16,15,11,10)(12,14)(17,19,22,21,23,18),
  (1,4,3,7,11,10,9,13)(2,8,15,6,12,14,5,16)(17,20,19,22,21,24,23,18),
  (1,4,16,15,11,10,6,5)(2,8,13,3,12,14,7,9)(17,19,24,22,21,23,20,18),
  (1,4,16,13)(2,8,15,9)(3,12,14,5)(6,7,11,10)(17,18)(19,24,23,20)(21,22),
  (1,4,8,5)(2,16,7,9)(3,12,6,13)(10,14,15,11)(17,19,21,23)(18,20)(22,24),
  (1,4,8,7,11,10,14,13)(2,16,5,3,12,6,15,9)(17,18,20,19,21,22,24,23),
  (1,4,3,15,11,10,9,5)(2,16,7,14,12,6,13,8)(17,19,22,24,21,23,18,20),
  (1,4,3,15,14,13)(2,16)(5,8,7,11,10,9)(6,12)(18,20,19,22,24,23),
  (1,4,24,3,2,21)(5,23,6,15,19,16)(7,18,14,13,22,8)(9,12,17,11,10,20),
  (1,4,22,8,12,17,11,10,18,14,2,21)(3,13,24,16,5,23,9,7,20,6,15,19),
  (1,4,22,8,5,24)(2,21,16,12,17,6)(3,13,23,9,7,19)(10,18,14,15,20,11),
  (1,4,23,11,10,19)(2,21,3,15,18,14)(5,22,8,12,17,9)(6,13,24,16,7,20),
  (1,4,23,11,10,19)(2,21,16,12,17,6)(3,15,20,9,5,24)(7,18,14,13,22,8),
  (1,4,24,8,7,17,11,10,20,14,13,21)(2,22,3,5,23,6,12,18,9,15,19,16),
```

```
  (1,4,23,11,10,19)(2,22,3,12,18,9)(5,21,8,15,17,14)(6,13,24,16,7,20),
  (1,4,23,11,10,19)(2,22,3,12,18,9)(5,24,8,7,17,6)(13,21,16,15,20,14),
  (1,4,21,16,15,20,11,10,17,6,5,24)(2,22,3,13,23,14,12,18,9,7,19,8),
  (1,4,24,3,7,18,11,10,20,9,13,22)(2,23,6,15,17,14,12,19,16,5,21,8),
  (1,4,24,8,5,22)(2,23,6,12,19,16)(3,7,17,9,13,21)(10,20,14,15,18,11),
  (1,4,21,3,15,18,11,10,17,9,5,22)(2,23,14,7,20,6,12,19,8,13,24,16),
  (1,4,21,16,13,22)(2,23,14,12,19,8)(3,15,20,9,5,24)(6,7,18,11,10,17),
  (1,4,21,3,2,24)(5,22,16,15,18,6)(7,19,8,13,23,14)(9,12,20,11,10,17),
  (1,4,22,16,12,20,11,10,18,6,2,24)(3,5,21,8,13,23,9,15,17,14,7,19),
  (1,4,22,16,13,21)(2,24,8,12,20,14)(3,5,23,9,15,19)(6,7,17,11,10,18),
  (1,4,23,11,10,19)(2,24,3,7,18,6)(5,21,8,15,17,14)(9,13,22,16,12,20),
  (1,4,23,11,10,19)(2,24,8,12,20,14)(3,7,17,9,13,21)(5,22,16,15,18,6) ]
gap> Length(pHel);
36
gap> #Orders of each symmetry in PHel.
gap> PHel:=pHel;;
gap> OrdH:=[];;
gap> for j in [1..Length(pHel)] do
>       OrdH[j]:=Order(pHel[j]);
> od;
gap> OrdH;
[ 4, 4, 6, 8, 8, 6, 4, 6, 6, 4, 6, 8, 8, 4, 4, 8, 8, 6, 6, 12, 6, 6,
  6, 12, 6, 6, 12, 12, 6, 12, 6, 6, 12, 6, 6, 6 ]
```

Note: The code used to classify conjugate symmetries is analogous to that presented for the 16-cell.

A.3.2 Structures of type $\mathsf{P}(S_1, S_2; v)$ with skew faces that are not polyhedra

The base vertex of the 24-cell is $v = 11$:

```
gap> v:=11;
11
```

Define $S_1 := R_0R_1R_2$.

```
gap> S1:=r0*r1*r2;
(1,24,2,3,21,4)(5,6,18,15,16,22)(7,14,23,13,8,19)(9,17,10,11,20,12)
```

Define S_2 by $S_2 := R_2R_3R_1$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r2*r3*r1;
(2,22)(3,16,14,9,6,8)(4,21,13,19,5,24)(7,23,15,20,10,17)(12,18)
gap> Order(S1*S2);
2
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
24
gap> H:=Group(S2);;
gap> Index(G, H);
192
```

Define S_2 by $S_2 := R_2R_1R_2R_3R_2$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r2*r1*r2*r3*r2;
(2,24,4,22,13,19)(3,8,16,9,14,6)(5,21)(7,23,12,20,10,18)(15,17)
gap> Order(S1*S2);
```

```
2
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
24
gap> H:=Group(S2);;
gap> Index(G, H);
192
```

Define $S_1 := R_0R_1R_2R_1R_3$.

```
S1:=r0*r1*r2*r1*r3;
(1,13,14,15,3,4)(2,16)(5,9,10,11,7,8)(6,12)(18,23,24,22,19,20)
```

Define S_2 by $S_2 := R_3(R_2R_1)^2$ and compute $|vG|$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r3*(r2*r1)^2;
(2,5,13,4)(3,6,9,16)(7,10,12,15)(8,14)(17,20,23,18)(19,22,21,24)
gap> Order(S1*S2);
2
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
16
gap> H:=Group(S2);;
gap> Index(G, H);
96
```

Define S_2 by $S_2 := R_3R_2R_1R_2R_3R_1R_2$ and compute $|vG|$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r3*r2*r1*r2*r3*r1*r2;
(2,13,4,5)(3,9)(6,14,16,8)(7,10,15,12)(17,18,20,23)(19,21,22,24)
gap> Order(S1*S2);
2
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
16
gap> H:=Group(S2);;
gap> Index(G, H);
96
```

A.3.3 Structures of type $\mathsf{P}(S_1, S_2; v)$ with helical faces that are not polyhedra

Define $S_1 := R_0R_1R_2R_3$.

```
gap> S1:=r0*r1*r2*r3;
(1,22,5,9,17,10,11,18,15,3,21,4)(2,16,24,13,8,19,12,6,20,7,14,23)
```

Define S_2 by $S_2 := R_3R_2R_1R_2$ and compute $|vG|$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r3*r2*r1*r2;
(2,19,13,21)(3,6,9,16)(4,24,5,22)(7,17,12,23)(10,20,15,18)
gap> Order(S1*S2);
2
```

```
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
24
gap> H:=Group(S2);;
gap> Index(G, H);
48
```

Define S_2 by $S_2 := R_2R_3R_2R_1$ and compute $|vG|$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r2*r3*r2*r1;
(2,19,5,24)(3,14,9,8)(4,21,13,22)(7,18,10,17)(12,23,15,20)
gap> Order(S1*S2);
2
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
24
gap> H:=Group(S2);;
gap> Index(G, H);
48
```

A.4 120-cell

A.4.1 Cycles of the 120-cell

The base vertex and distinguished generators of the 120-cell are based on the article [2], appear in [1], and are as follows:

```
gap> rho := (1 + Sqrt(5)) / 2;; #Golden ratio.
gap> #Distinguished generators of the 120-cell as reflection matrices.
gap> r0 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, -1, 0],
>          [0, 0, 0, 1] ];;
gap> r1 := (1/2) * [ [2, 0, 0, 0],
>                    [0, rho, -1, rho - 1],
>                    [0, -1, 1 - rho, rho],
>                    [0, rho - 1, rho, 1] ];;
gap> r2 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, 1, 0],
>          [0, 0, 0, -1] ];;
gap> r3 := (1/2) * [ [rho, 1, 0, -1 + rho],
>                    [1, 1 - rho, 0, -rho],
>                    [0, 0, 2, 0],
>                    [-1 + rho, -rho, 0, 1] ];;
gap> v:=[1+rho,1,-2+rho,0];; #Base vertex.
gap> P:=Group(r0,r1,r2,r3);;
gap> eGC:=Elements(P);;
gap> e:=v*r0;;
gap> OrdC:=[];;
gap> for j in [1..Length(eGC)] do
```

```
>         if Order(eGC[j])>1 then
>                 Add(OrdC,Order(eGC[j]));
>         fi;
> od;
gap> Set(OrdC); #Lenght of the cycles.
[ 2, 3, 4, 5, 6, 10, 12, 15, 20, 30 ]
gap> # We will store the cycles that contain v and e (vertex and base edge) in
#Cycles.
gap> Ciclos:=[];;
gap> Simgc:=[];; #In Simgc we will store the symmetries candidate to be S1.
#We will remove the symmetries the order 3, as these correspond to the
#triangular faces of the 120-cell.
gap> for j in [1..Length(eGC)] do
>         ciclo:=[];
>         for i in [1..Order(eGC[j])] do
>                 Add(ciclo,v*eGC[j]^i);
>         od;
>         if Length(Set(ciclo))>3 and v*eGC[j]^-1=e and
 Length(Set(ciclo))=Order(eGC[j]) then
>                 Add(Ciclos,ciclo);
>                 Add(Simgc,eGC[j]);
>         fi;
> od;
gap> OCiclos:=[];;
gap> for j in [1..Length(Ciclos)] do
>         Add(OCiclos,Length(Ciclos[j]));
> od;
gap> Set(OCiclos); # Length of cycles containing v and e.
[ 5, 10, 30 ]
```

We group the preceding symmetries according to their order. As an illustrative case, we consider symmetries of order 5 and then compute their conjugacy class in order to obtain a representative:

```
gap> OSim:=5;;
gap> SimOrd:=[];; #Candidate for being an S1 of order 5.
gap> for j in [1..Length(Simgc)] do
>         if Order(Simgc[j])=OSim then
>                 Add(SimOrd,Simgc[j]);
>         fi;
> od;
gap> SimOrd;;
gap> Length(SimOrd);
3
gap> Conj:=[];;
gap>     for j in [1..Length(SimOrd)] do
>                 for k in [1..Length(eGC)] do
>                         if  eGC[k]*SimOrd[1]*eGC[k]^-1= SimOrd[j] then
>                                 Add(Conj,SimOrd[j]);
>                                 break;
>                         else
>                                 continue;
>                         fi;
>                 od;
>         od;
gap> Conj;; #This list contains the elements of the conjugation class of
#SimOrd[1].
```

```
gap> Length(Conj);
3
gap> #We select the second element from the previous list as our
#representative, which is:
 gap> Factorization(P,Conj[2]);
x1*x2
gap> #So, the candidate for an S1 of order 5 is as follows:
gap> s1:=r0*r1;;
```

Next, we determine the dimension of the affine subspace containing the vertices of the cycle. For this purpose, we define the following function **espacio**, with parameters a symmetry S and a vertex v.

```
gap> espacio:= function(S,v)
> local A, B, i;
> A:=[];
> for i in [0..Order(S)-1] do
>       Add(A,v*S^i);
> od;
> B:=[];
> for i in [1..Order(S)] do
>       Add(B,A[i]-v);
> od;
> return(RankMat(B));
> end;
function( S, v ) ... end

gap> s1:=r0*r1;; #Order 5.
gap> espacio(s1,v); #Plane (pentagonal faces of the 120-cell).
2
```

If there exists $g \in Stab_{[5,3,3]}(v)$ such that $gS_1g^{-1} = S$, where S is an element of the list Conj, then the cycles $v\langle S_1\rangle$ and $v\langle S\rangle$ are isometric.

```
gap> stabV:=Elements(Stabilizer(P,v));;
gap> cIsom1:=[];;
gap> ##Isometric cycles to cycle v<S1>.
gap> for j in [1..Length(Conj)] do
> for k in [1..Length(stabV)] do
>       if  stabV[k]*s1*stabV[k]^-1=  Conj[j] then
>               Add(cIsom1, [j,Factorization(P,stabV[k]),
Factorization(P,Conj[j])]);
>               break;
>       else
>               continue;
>       fi;
> od;
> od;
gap> cIsom1;
[ [ 1, x3, x1*x2*x3*x2 ], [ 2, <identity ...>, x1*x2 ],
  [ 3, x3*x4*x3, x1*x2*x3*x4*x3*x2 ] ]
```

The preceding list shows that the three elements of the conjugacy class are isometric. In this case, $S_1 := R_0R_1$. For each sublist, the second entry corresponds to g, and the third entry corresponds to S.

Analogously, we group the remaining candidate symmetries for S_1 according to their order, select a representative from each conjugacy class, and determine the dimension of the subspace containing the cycle constructed by the corresponding symmetry.

```
gap> s1:=r0*r1*r2;; #Order 10.
```

```
gap> espacio(s1,v); #Skew.
3
gap> Length(Conj);
6
gap> cIsom1;
[ [ 1, <identity ...>, x1*x2*x3 ], [ 2, x3, x3*x1*x2 ],
  [ 3, x4, x1*x2*x3*x4*x3 ], [ 4, x3*x4, x1*x2*x3*x4*x2 ],
  [ 5, x3*x4*x3, x4*x1*x2*x3*x2 ], [ 6, x4*x3, x3*x4*x3*x1*x2 ] ]

gap> s1:=r0*r1*r2*r3;; #Order 30.
gap> espacio(s1,v); #Helix.
4
gap> Length(Conj);
6
gap> cIsom1;
[ [ 1, x4, x4*x1*x2*x3 ], [ 2, x3, x1*x2*x3*x4*x2*x3 ],
  [ 3, x3*x4, x3*x4*x1*x2 ], [ 4, <identity ...>, x1*x2*x3*x4 ],
  [ 5, x4*x3, x3*x4*x1*x2*x3*x2 ], [ 6, x3*x4*x3, x4*x3*x1*x2 ] ]
```

A.4.2 Candidates for S_2 in the 120-cell for skew S_1

$\underline{S_1 = R_0R_1R_2.}$

```
gap> GVerFig:=Group(r1,r2,r3); #Group of the vertex figure of the 120-cell.
<matrix group with 3 generators>
gap> ElVF:=Elements(GVerFig);;
gap> #Define S1.
gap> S1:=r0*r1*r2;; #Order 10.
gap> #The candidates for S2 are those elements belonging to the vertex-figure
#group of the 120-cell; we exclude the identity element as well as elements of
#order 2 and 3, since these symmetries represent digons —orin the case of
#elements of order —3generate cycles that lie within a single plane.
gap> CandSigma2:=[];; #Candidatos a ser S2.
gap> for j in [1..Length(ElVF)] do
>       if (Order(ElVF[j])=1 or Order(ElVF[j])=2 or
 Order(ElVF[j])=3)=true then
>               continue;
>       else
>               Add(CandSigma2,ElVF[j]);
>       fi;
> od;
gap> CandSigma2;;
gap> Length(CandSigma2);
6
gap> #Among the potential candidates for S2, we will exclude those that
 #do not meet the following condition: Order(S1*S2)=2;
gap> S2:=[];;
gap> for j in [1..Length(CandSigma2)] do
>       if Order(S1*CandSigma2[j])=2 then
>               Add(S2,Factorization(P,CandSigma2[j]));
>       fi;
> od;
gap> S2;
[ x2*x3*x4*x2*x3, x3*x4*x2 ]
```

```
gap> u:=v*S1;;

gap> S2:=r1*r2*r3*r1*r2;;
gap> Order(S2);
4
gap> espacio(S2,u); #Skew.
3

gap> S2:=r2*r3*r1;;
gap> Order(S2);
4
gap> espacio(S2,u); #Skew.
3
```

A.4.3 Structures of type $\mathsf{P}(S_1, S_2; v)$ with skew faces that are not polyhedra

Define $S_1 := R_0R_1R_2$.

```
gap> S1:=r0*r1*r2;;
```

Define S_2 by $S_2 := R_2R_3R_1$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r2*r3*r1;;
gap> G:=Group(S1,S2);
<matrix group with 2 generators>
gap> Size(Orbit(G, v, OnPoints));
600
gap> H:=Group(S2);;
gap> Index(G, H);
3600
```

Define S_2 by $S_2 := R_1R_2R_3R_1R_2$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=r1*r2*r3*r1*r2;;
gap> G:=Group(S1,S2);
<matrix group with 2 generators>
gap> Size(Orbit(G, v, OnPoints));
600
gap> H:=Group(S2);;
gap> Index(G, H);
3600
```

A.4.4 Candidates for S_2 in the 120-cell for helical S_1

$\underline{S_1 = R_0R_1R_2R_3.}$

```
gap> S1:=r0*r1*r2*r3;; #Order 30.
gap> CandSigma2:=[];; #Candidates to be S2.
gap> for j in [1..Length(ElVF)] do
>       if (Order(ElVF[j])=1 or Order(ElVF[j])=2)=true then
>               continue;
>       else
>               Add(CandSigma2,ElVF[j]);
>       fi;
```

```
> od;
gap> CandSigma2;;
gap> Length(CandSigma2);
14
gap> #Among the potential candidates for S2, we will exclude those who do not
#satisfy the following condition: Order(S1*S2)=2;
gap> S2:=[];;
gap> for j in [1..Length(CandSigma2)] do
>       if Order(S1*CandSigma2[j])=2 then
>               Add(S2,Factorization(P,CandSigma2[j]));
>       fi;
> od;
gap> S2;
[ x4*x2*x3*x2, x3*x4*x3*x2, x4*x3*x2 ]

gap> u:=v*S1;;

gap> S2:=r3*r1*r2*r1;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=r2*r3*r2*r1;;
gap> Order(S2);
3
gap> espacio(S2,u);  #Plane.
2

gap> S2:=r3*r2*r1;;
gap> Order(S2);
4
gap> espacio(S2,u); #Skew.
3
```

Search for the polytope of type $\{30,3\}$in the list [3].

The group of the first polyhedron appears in [3] listed as: R97.9: Type $\{3,30\}_{20}$ of order 2880. We denote the group of this polyhedron by L.

```
gap> l:= FreeGroup( "l0", "l1","l2" );;
gap> L:=l/[ l.1^2, l.2^-3, (l.2 * l.3)^2, (l.2 * l.1)^2, (l.3 * l.1)^2,
l.3 * l.2 * l.3^-2 * l.2 * l.3^-2 * l.2 * l.3^-1 * l.2 * l.3^-2 * l.2
* l.3^2 * l.2^-1 * l.3^2, l.3 * l.2 * l.3^-3 * l.2^-1
* l.3^5 * l.2^-1 * l.3^-3 * l.2 * l.3^6 ];
<fp group on the generators [ l0, l1, l2 ]>
gap> Size(L);
2880
gap> cc:=ConjugacyClassesSubgroups(L);;
rs1:=[];; #The subgroups of L of order 1440 will be stored here.
gap> for i in [1..Length(cc)] do
>       if Size(Representative(cc[i]))=1440 then
>               Add(rs1,Representative(cc[i]));
>       fi;
```

```
> od;
gap> Length(rs1); #There are three subgroups of L of order 1440.
3
```

Group of the second polyhedron of type $\{30,3\}$, which appears in [3] listed as follows:
R97.10: Type $\{3,30\}_{40}$ of order 2880. We denote the group of this polyhedron by F.

```
gap> #R97.10 :  Type {3,30}_40  Order 2880  mV = 2  mF = 1
gap> f:= FreeGroup( "f0", "f1","f2" );;
gap> F:=f/[ f.3^2, f.1^-3, (f.1 * f.2)^2, (f.1 * f.3)^2, (f.2 * f.3)^2,
 (f.2^-2 * f.1 * f.2^-1)^3 ];
<fp group on the generators [ f0, f1, f2 ]>
gap> Size(F);
2880
gap> ccs:=ConjugacyClassesSubgroups(F);;
gap> rs2:=[];; #The subgroups of F of order 1440 will be stored here..
gap> for i in [1..Length(ccs)] do
>       if Size(Representative(ccs[i]))=1440 then
>               Add(rs2,Representative(ccs[i]));
>       fi;
> od;
gap> Length(rs2); #There is only one subgroup of F of order 1440.
1
```

G is the symmetry group of the 120-cell.

```
gap> g:=FreeGroup("r0","r1","r2","r3");
<free group on the generators [ r0, r1, r2, r3 ]>
gap> G:=g/[g.1^2,g.2^2,g.3^2,g.4^2,(g.1*g.2)^5,(g.2*g.3)^3,(g.3*g.4)^3,
(g.1*g.3)^2,(g.1*g.4)^2,(g.2*g.4)^2];
<fp group on the generators [ r0, r1, r2, r3 ]>
gap> Order(G);
14400
gap> r0:=G.1;;
gap> r1:=G.2;;
gap> r2:=G.3;;
gap> r3:=G.4;;
```

Next, we define S_1 and S_2, and we store the symmetry group of the polyhedron $\mathsf{P}(S_1,S_2;v)$ in `grup303`.

```
gap> s1:=r0*r1*r2*r3;
r0*r1*r2*r3
gap> s2:=r3*r2*r1*r3;
r3*r2*r1*r3
gap> grup303:=Group(s1,s2);
Group([ r0*r1*r2*r3, r3*r2*r1*r3 ])
```

We now test whether any of the subgroups of L (R97.9) of order 1440 is isomorphic to the symmetry group of the polyhedron $\mathsf{P}(S_1,S_2)$.

```
gap> LoadPackage("sonata");
gap> IsIsomorphicGroup(rs1[1],grup303);
false
gap> IsIsomorphicGroup(rs1[2],grup303);
false
gap> IsIsomorphicGroup(rs1[3],grup303);
false
```

We now test whether the unique subgroup of F (R97.10) of order 1440 is isomorphic to the symmetry group of the polyhedron $\mathsf{P}(S_1,S_2)$.

```
gap> IsIsomorphicGroup(rs2[1],grup303);
true
```

Therefore, the rotation subgroup of F (R97.10) is isomorphic to the group of $\mathsf{P}(S_1, S_2)$.

A.5 600-cell

A.5.1 Cycles of the 600-cell

The base vertex and distinguished generators of the 600-cell are based on the article [2], appear in [1], and are as follows:

```
gap> rho := (1 + Sqrt(5)) / 2;; #Razón dorada.
gap> #Generadores distinguidos del 600−celda como matrices de reflexión.
gap> R0 := (1/2) * [ [rho, 1, 0, −1 + rho],
>                    [1, 1 − rho, 0, −rho],
>                    [0, 0, 2, 0],
>                    [−1 + rho, −rho, 0, 1] ];;
gap>
gap> R1 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, 1, 0],
>          [0, 0, 0, −1] ];;
gap>
gap> R2 := (1/2) * [ [2, 0, 0, 0],
>                    [0, rho, −1, rho − 1],
>                    [0, −1, 1 − rho, rho],
>                    [0, rho − 1, rho, 1] ];;
gap>
gap> R3 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, −1, 0],
>          [0, 0, 0, 1] ];;
gap> v:=[1,0,0,0];; #Base vertex.
gap> P:=Group(R0,R1,R2,R3);;
gap> eGC:=Elements(P);;
gap> e:=v*R0;;
gap> OrdC:=[];;
gap> for j in [1..Length(eGC)] do
>       if Order(eGC[j])>1 then
>               Add(OrdC,Order(eGC[j]));
>       fi;
> od;
gap> Set(OrdC); #Length of the cycles.
[ 2, 3, 4, 5, 6, 10, 12, 15, 20, 30 ]
gap> #We will store the cycles that contain v and e (the base vertex and edge)
#in Ciclos.
gap> Ciclos:=[];;
gap> Simgc:=[];; #Symetries candidates to be S1.
gap> for j in [1..Length(eGC)] do
>       ciclo:=[];
>       for i in [1..Order(eGC[j])] do
>               Add(ciclo,v*eGC[j]^i);
>       od;
>       if Length(Set(ciclo))>3 and v*eGC[j]^−1=e and
```

```
                    Length(Set(ciclo))=Order(eGC[j]) then
>                 Add(Ciclos,ciclo);
>                 Add(Simgc,eGC[j]);
>         fi;
> od;
gap> OCiclos:=[];;
gap> for j in [1..Length(Ciclos)] do
>         Add(OCiclos,Length(Ciclos[j]));
> od;
gap> Set(OCiclos); #Length of the cycles that contain v and e.
[ 4, 5, 6, 10, 12, 15, 20, 30 ]
gap> #Grouping the symmetries according to their order.
gap> #We will take the symmetries of order 4 as an example, and then
#determine their conjugacy class to obtain a representative:
gap> OSim:=4;;
gap> SimOrd:=[];; #Candidate to be S1 of orden 4.
gap> for j in [1..Length(Simgc)] do
>         if Order(Simgc[j])=OSim then
>                 Add(SimOrd,Simgc[j]);
>         fi;
> od;
gap> SimOrd;;
gap> Length(SimOrd);
10
gap> Conj:=[];;
gap>     for j in [1..Length(SimOrd)] do
>                 for k in [1..Length(eGC)] do
>                         if  eGC[k]*SimOrd[1]*eGC[k]^-1= SimOrd[j] then
>                                 Add(Conj,SimOrd[j]);
>                                 break;
>                         else
>                                 continue;
>                         fi;
>                 od;
>         od;
gap>Conj;; #This list contains the elements of the conjugacy class of
#SimOrd[1].
gap>Length(Conj);
10
gap> #We have selected the seventh element from the previous list as our
#representative, which is:
gap> Factorization(P,Conj[7]);
x1*x2*x3
gap> #So, the candidate for an S1 of order 4 is as follows:
gap> s1:=R0*R1*R2;; #Orden 4
```

We determine the dimension of the affine subspace containing the vertices of the cycle. For this purpose, we define the following function **espacio**, with parameters S and v.

```
gap> espacio:= function(S,v)
> local A, B, i;
> A:=[];
> for i in [0..Order(S)-1] do
>         Add(A,v*S^i);
> od;
> B:=[];
```

```
> for i in [1..Order(S)] do
>         Add(B,A[i]-v);
> od;
> return(RankMat(B));
> end;
function( S, v ) ... end

gap> s1:=R0*R1*R2;; #Order 4
gap> espacio(s1,v); # Skew (This corresponds to a Petrie polygon in the base
#tetrahedron).
3
```

If g exists in $Stab_{[5,3,3]}(v)$, such that $gS1g^{-1} = S$, where S is an element of the list Conj, then the cycles $v\langle S_1\rangle$ and $v\langle S\rangle$ are isometric.

```
gap> cIsom1:=[];;
gap> #Isometric cycles to the cycle v<S1>.
gap> for j in [1..Length(Conj)] do
> for k in [1..Length(stabV)] do
>         if  stabV[k]*s1*stabV[k]^-1=  Conj[j] then
>                 Add(cIsom1, [j,Factorization(P,stabV[k]),Factorization(P,Conj[j])]);
>                 break;
>         else
>                 continue;
>         fi;
> od;
> od;
gap> cIsom1;
[ [ 1, x3*x4, x3*x4*x1*x2*x3*x4*x3 ],
  [ 2, (x3*x4)^2*x3, x4*x3*x4*x1*x2*x3*x4*x2*x3 ],
  [ 3, x4*x3*x4, x4*x1*(x2*x3*x4)^2*x3 ],
  [ 4, (x3*x4)^2, x3*x4*x1*(x2*x3*x4)^2 ], [ 5, x3, x3*x1*x2 ],
  [ 6, x4*x3, x4*x3*x4*x1*x2 ], [ 7, <identity ...>, x1*x2*x3 ],
  [ 8, x3*x4*x3, (x3*x4)^2*x1*x2*x3 ], [ 9, x4, x4*x1*x2*x3*x4 ],
  [ 10, (x4*x3)^2, (x3*x4)^2*x1*x2*x3*x4*x2 ] ]
```

The list above indicates that the ten elements of the conjugacy class are isometric. In this case, $S_1 := R_0R_1R_2$. Within each sublist, the second element corresponds to g, and the third element corresponds to S.

Analogously, we classify the remaining symmetries—those that are candidates for S_1—based on their order, and we select a representative from each conjugacy class. We then determine the space in which the cycle that constructs the corresponding symmetry resides. In some instances, there is more than one conjugacy class, and within each such class, we categorize the cycles that are isometric to one another.

```
gap> s1:=R0*R1*R2*R3*R2*R1;; #Order 5
gap> espacio(s1,v); #Plane.
2
gap> Length(Conj);
5
gap> cIsom1;
[ [ 1, (x3*x4)^2, x3*x4*x1*(x2*x3*x4)^3*x2*x3 ],
  [ 2, <identity ...>, x1*x2*x3*x4*x3*x2 ],
  [ 3, x4, x1*x2*(x3*x4)^2*x3*x2 ],
  [ 4, x3*x4*x3, x1*(x2*x3*x4)^2*x2*x3*x2 ],
  [ 5, (x4*x3)^2, x4*x1*(x2*x3*x4)^3*x2 ] ]

gap> s1:=R3*R0*R1*R2*R3*R2*R1;; #Order 6
gap> espacio(s1,v); #Skew.
```

```
3
gap> Length(Conj);
10
gap> cIsom1;
[ [ 1, (x3*x4)^2*x3, x4*x1*(x2*x3*x4)^3*x2*x3 ],
  [ 2, (x3*x4)^2, x3*x4*x1*(x2*x3*x4)^3*x3 ],
  [ 3, <identity ...>, x4*x1*x2*x3*x4*x3*x2 ],
  [ 4, x3, x3*x4*x1*x2*x3*x4*x2*x3*x2 ],
  [ 5, x4, x1*x2*(x3*x4)^2*x2 ],
  [ 6, x4*x3, x4*x3*x4*x1*(x2*x3*x4)^2*x2 ],
  [ 7, x3*x4*x3, x4*x3*x4*x1*(x2*x3*x4)^2*x2*x3*x2 ],
  [ 8, x3*x4, x1*(x2*x3*x4)^2*x2*x3 ],
  [ 9, (x4*x3)^2, x3*x4*x1*(x2*x3*x4)^3*x2 ],
  [ 10, x4*x3*x4, x4*x1*(x2*x3*x4)^3 ] ]
```

There are four conjugacy classes for symmetries of order 10. Their respective representatives are s1, sa, sb, and sc.

```
gap> s1:=R0*R1*R2*R3*R1;; #Order 10
gap> A:=[];;
gap> for i in [0..Order(s1)-1] do
>       Add(A,v*s1^i);
> od;
gap> Length(Conj);
25
```

For this first class of conjugation with 25 elements, we have three isometric classes:

```
gap> s1:=R0*(R1*R2*R3)^3*R1*R2*R1;;
gap> espacio(s1,v); #Plano.
2
gap> A11:=[];;
gap> for i in [0..Order(s1)-1] do
>       Add(A11,v*s1^i);
> od;
gap> cIsom1; #First isometric class.
[ [ 1, <identity ...>, x1*(x2*x3*x4)^3*x2*x3*x2 ],
  [ 2, x4, x4*x3*x4*x1*(x2*x3*x4)^3*x3*x2 ],
  [ 3, (x3*x4)^2, x1*x2*(x3*x4)^2*x2*x3*x4*x3*x2 ],
  [ 4, (x3*x4)^2*x3, x4*x1*(x2*x3*x4)^3*x3*x2 ],
  [ 5, x3*x4*x3, x3*x4*x1*(x2*x3*x4)^3*x2*x3*x2 ] ]
gap> Length(cIsom1);
5

gap> s1:=R3*R2*R3*R0*R1*R2*R3*R2*R1;;
gap> espacio(s1,v); #Skew.
3
gap> A22:=[];;
gap> for i in [0..Order(s1)-1] do
>       Add(A22,v*s1^i);
> od;
gap> cIsom1; #Second isometric class.
[ [ 6, (x3*x4)^2*x3, x1*x2*x3*x4*(x3*x4*x2)^2*x3 ],
  [ 7, (x3*x4)^2, x1*(x2*x3*x4)^3*x3 ],
  [ 8, x3, (x3*x4)^2*x1*x2*x3*x4*x2*x3*x2 ],
  [ 9, <identity ...>, x4*x3*x4*x1*x2*x3*x4*x3*x2 ],
  [ 12, x4*x3, x3*x4*x1*(x2*x3*x4)^2*x3*x2 ],
```

```
  [ 13, x4, x1*(x2*x3*x4)^2*x3*x2 ],
  [ 18, x3*x4*x3, x4*x1*(x2*x3*x4)^2*x2*x3*x2 ],
  [ 19, x3*x4, x1*x2*(x3*x4)^2*x2*x3*x2 ],
  [ 20, (x4*x3)^2, x1*(x2*x3*x4)^3*x2 ],
  [ 21, x4*x3*x4, x1*x2*(x3*x4)^2*x2*x3*x4 ] ]
gap> Length(cIsom1);
10

gap> s1:=R0*R1*R2*R3*R1;;
gap> espacio(s1,v); #Skew.
3
gap> cIsom1; #Third isometric class.
[ [ 10, x3, x1*x2*x3*x4*x3 ],
  [ 11, (x4*x3)^2, x4*x1*x2*x3*x4*x2*x3 ],
  [ 14, x4*x3, x1*x2*(x3*x4)^2*x3 ],
  [ 15, x3*x4*x3, x1*(x2*x3*x4)^2 ],
  [ 16, x3*x4, x3*x4*x3*x1*x2 ],
  [ 17, x4*x3*x4, (x3*x4)^2*x3*x1*x2 ],
  [ 22, (x3*x4)^2, x4*x3*x4*x1*x2*x3*x2 ],
  [ 23, x4, x4*x1*x2*x3*x2 ],
  [ 24, (x3*x4)^2*x3, x3*x4*x1*x2*x3*x4*x2 ],
  [ 25, <identity ...>, x1*x2*x3*x4*x2 ] ]
gap> Length(cIsom1);
10
```

The second conjugacy class of symmetries of order 10 contains two symmetries, with *sa* as its representative. Furthermore, the cycles they induce are isometric.

```
gap> sa:=R3*R0*(R1*R2*R3)^3*R1*R2*R1;;
gap> A1:=[];;
gap> for i in [0..Order(s1)-1] do
>       Add(A1,v*sa^i);
> od;
gap>
gap> espacio(sa,v); #Plane.
2
gap> cIsom1;
[ [ 1, (x4*x3)^2, x4*x1*(x2*x3*x4)^3*x2*x3*x2 ],
  [ 2, x3*x4*x3, x3*x4*x1*(x2*x3*x4)^3*x3*x2 ] ]
```

The third conjugacy class of symmetries of order 10 contains two symmetries, with *sb* as its representative. Furthermore, the cycles they induce are isometric.

```
gap> sb:=R0*(R1*R2*R3)^3*R2*R1;;
gap> B1:=[];;
gap> for i in [0..Order(s1)-1] do
>       Add(B1,v*sb^i);
> od;
gap> espacio(sb,v); #Plane.
gap> cIsom1;
[ [ 1, (x4*x3)^2, x1*x2*x3*x4*(x3*x4*x2)^2*x3*x2 ],
  [ 2, x3*x4*x3, x1*(x2*x3*x4)^3*x3*x2 ] ]
```

The fourth conjugacy class of symmetries of order 10 has only one element, which is sc.

```
gap> sc:=R3*R2*R3*R0*(R1*R2*R3)^3*R1*R2*R1;;
gap> C1:=[];;
gap> for i in [0..Order(s1)-1] do
```

```
>          Add(C1,v*sc^i);
> od;
gap> espacio(sc,v); #Plane.
```

Next, we observe that the skew cycles are distinct, since they do not share the same vertex set. In contrast, all planar cycles share the same vertex set.

```
gap> Set(A)=Set(A22);
false
gap> Set(A1)=Set(A11);
true
gap> Set(A1)=Set(B1);
true
gap> Set(B1)=Set(C1);
true
```

There is only one conjugacy class (consisting of 10 elements) of the symmetries of interest to us—those of order 12; all the cycles are isometric and induce helical cycles.

```
gap> s1:=R0*(R1*R2*R3)^3;; #Order 12
gap> espacio(s1,v); #Helix.
4
gap> cIsom1;
[ [ 1, x3, x1*x2*(x3*x4)^2*x2*x3*x4*x3 ],
  [ 2, x4*x3, x4*x1*(x2*x3*x4)^3*x3 ],
  [ 3, (x3*x4)^2, (x3*x4)^2*x1*x2*x3*x4*x3*x2 ],
  [ 4, x4*x3*x4, x4*x3*x4*x1*x2*x3*x4*x2*x3*x2 ],
  [ 5, (x3*x4)^2*x3, x4*x3*x4*x1*(x2*x3*x4)^2*x3*x2 ],
  [ 6, x3*x4, x3*x4*x1*(x2*x3*x4)^2*x2 ],
  [ 7, (x4*x3)^2, x4*x3*x4*x1*(x2*x3*x4)^2*x2*x3 ],
  [ 8, x4, x4*x1*(x2*x3*x4)^2*x2*x3 ],
  [ 9, x3*x4*x3, x3*x4*x1*(x2*x3*x4)^3 ],
  [ 10, <identity ...>, x1*(x2*x3*x4)^3 ] ]
```

There is only one conjugacy class (consisting of 10 elements) of the symmetries of interest to us that are of order 15; all the cycles are isometric and induce helical cycles.

```
gap> s1:=R2*R3*R0*R1*R2*R3*R2*R1;; #Order 15
gap> espacio(s1,v); #Helix.
4
gap> cIsom1;
[ [ 1, (x3*x4)^2*x3, x1*(x2*x3*x4)^3*x2*x3 ],
  [ 2, (x3*x4)^2, x4*x3*x4*x1*(x2*x3*x4)^3*x3 ],
  [ 3, <identity ...>, x3*x4*x1*x2*x3*x4*x3*x2 ],
  [ 4, x3, x4*x1*x2*x3*x4*x2*x3*x2 ],
  [ 5, x4, x4*x1*(x2*x3*x4)^2*x3*x2 ],
  [ 6, x4*x3, x1*(x2*x3*x4)^2*x2 ],
  [ 7, x3*x4, x3*x4*x1*(x2*x3*x4)^2*x2*x3*x2 ],
  [ 8, x3*x4*x3, x1*x2*(x3*x4)^2*x2*x3 ],
  [ 9, x4*x3*x4, x4*x3*x4*x1*(x2*x3*x4)^3*x2 ],
  [ 10, (x4*x3)^2, x1*x2*x3*x4*(x3*x4*x2)^2 ] ]
```

There is only one conjugacy class (containing 10 elements) of the symmetries of interest to us—those of order 20; all the cycles are isometric and induce helical cycles.

```
gap> s1:=R2*R3*R0*R1*R2*R3;; #Order 20
gap> espacio(s1,v); #Helix.
4
gap> cIsom1;
```

```
[ [ 1, (x4*x3)^2, x4*x3*x4*x1*x2*x3*x4*x3 ],
  [ 2, x3, x4*x1*x2*x3*x4*x3 ],
  [ 3, x3*x4*x3, x1*(x2*x3*x4)^2*x3 ],
  [ 4, x4*x3, x1*x2*(x3*x4)^2 ],
  [ 5, x4*x3*x4, (x4*x3)^2*x1*x2 ],
  [ 6, x3*x4, (x3*x4)^2*x1*x2 ],
  [ 7, x4, x4*x3*x4*x1*x2*x3 ],
  [ 8, (x3*x4)^2, (x3*x4)^2*x1*x2*x3*x2 ],
  [ 9, <identity ...>, x3*x4*x1*x2*x3*x4 ],
  [ 10, (x3*x4)^2*x3, (x3*x4)^2*x1*x2*x3*x4 ] ]
gap> Length(cIsom1);
```

There is only one conjugacy class (consisting of 10 elements) of the symmetries of interest to us—those of order 30; all the cycles are isometric and induce helical cycles.

```
gap> s1:=R0*R1*R2*R3;; #Order 30
gap> espacio(s1,v); #Helix.
4
gap> cIsom1;
[ [ 1, (x4*x3)^2, (x3*x4)^2*x1*x2*x3*x4*x2*x3 ],
  [ 2, x3, x1*x2*x3*x4*x2*x3 ],
  [ 3, x3*x4*x3, x3*x4*x1*(x2*x3*x4)^2*x3 ],
  [ 4, x4*x3, x4*x1*(x2*x3*x4)^2 ],
  [ 5, x4*x3*x4, x4*x3*x1*x2 ],
  [ 6, x3*x4, x3*x4*x1*x2 ],
  [ 7, (x3*x4)^2, x3*x4*x1*x2*x3*x2 ],
  [ 8, x4, x4*x1*x2*x3 ],
  [ 9, (x3*x4)^2*x3, x4*x3*x4*x1*x2*x3*x4*x2 ],
  [ 10, <identity ...>, x1*x2*x3*x4 ] ]
```

A.5.2 Candidates for S_2 in the 600-cell for skew S_1

$\underline{S_1 = R_0R_1R_2.}$

```
gap> GVerFig:=Group(R1,R2,R3); #Group of the vertex figure of the 600-cell.
<matrix group with 3 generators>
gap> ElVF:=Elements(GVerFig);;
gap> #Defining S1 as S1.
gap> S1:=R0*R1*R2;; #Order 4.
gap> CandSigma2:=[];;
gap> for j in [1..Length(ElVF)] do
>       if (Order(ElVF[j])=1 or Order(ElVF[j])=2 or
 Order(ElVF[j])=3)=true then
>               continue;
>       else
>               Add(CandSigma2,ElVF[j]);
>       fi;
> od;
gap> CandSigma2;;
gap> Length(CandSigma2);
68
gap> S2:=[];;
gap> for j in [1..Length(CandSigma2)] do
>       if Order(S1*CandSigma2[j])=2 then
>               Add(S2,Factorization(P,CandSigma2[j]));
```

```
>          fi;
> od;
gap> #The following symmetries are the candidates to be S2 for S1=R0*R1*R2.
gap> S2;
[ (x2*x3*x4)^2*x3, x3*x4*x2, x3*x4*x2*x3*x4, x2*x3*x4*x2*x3 ]

gap> u:=v*S1;;

gap> S2:=(R1*R2*R3)^2*R2;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew.
3

gap> S2:=R2*R3*R1;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew.
3

gap> S2:=R2*R3*R1*R2*R3;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew.
3

gap> S2:=R1*R2*R3*R1*R2;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew.
3
```

$S_1 = R_3R_0R_1R_2R_3R_2R_1$.

We repeat the previous code to obtain the candidates for S_2.

```
S1:=R3*R0*R1*R2*R3*R2*R1;; #Orden 6.
#Las siguientes simetrías son candidatas a ser S2 para
#S1=R3*R0*R1*R2*R3*R2*R1.
gap> S2;
[ x2*(x3*x4)^2*x2*x3*x4*x3, x2*(x3*x4)^2*x2*x3, x2*(x3*x4)^2*x2,
  (x2*x3*x4)^3,(x2*x3*x4)^2*x2 ]

gap> u:=v*S1;;

gap> S2:=R1*(R2*R3)^2*R1*R2*R3*R2;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew.
3

gap> S2:=R1*(R2*R3)^2*R1*R2;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew.
```

```
3

gap> S2:=R1*(R2*R3)^2*R1;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2

gap> S2:=(R1*R2*R3)^3;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew.
3

gap> S2:=(R1*R2*R3)^2*R1;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew.
3
```

$\underline{S_1 = R_3R_2R_3R_0R_1R_2R_3R_2R_1.}$

We repeat the previous code to obtain the candidates for S_2.

```
gap> S1:=R3*R2*R3*R0*R1*R2*R3*R2*R1;; #Orden 10 skew.
gap> S2;
[ x2*(x3*x4)^2*x2*x3*x4*x3, x2*(x3*x4)^2*x2*x3, x2*(x3*x4)^2*x2*x3*x4,
  (x2*x3*x4)^3, (x2*x3*x4)^2*x2 ]

gap> u:=v*S1;;

gap> S2:=R1*(R2*R3)^2*R1*R2*R3;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane
2

gap> S2:=R1*(R2*R3)^2*R1*R2;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew
3

gap> S2:=(R1*R2*R3)^2*R1;;
gap> Order(S2); #6
6
gap> espacio(S2,u); #Skew
3

gap> S2:=R1*(R2*R3)^2*R1*R2*R3*R2;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew
3
```

```
gap> S2:=(R1*R2*R3)^3;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew
```

$\underline{S_1 = R_0R_1R_2R_3R_1.}$
We repeat the previous code to obtain the candidates for S_2.

```
gap> S1:=R0*R1*R2*R3*R1;; #Order 10, skew.
gap> S2;
[ x4*x3*x4*x2*x3*x4*x3, x4*x2*x3*x2, x4*x3*x2, (x4*x2*x3)^2*x4,
  x4*x3*x4*x2*x3 ]

gap> u:=v*S1;;

gap> S2:=R3*R2*R3*R1*R2*R3*R2;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew
3

gap> S2:=R3*R1*R2*R1;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane
2

gap> S2:=R3*R2*R1;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew
3

gap> S2:=(R3*R1*R2)^2*R3;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew
3

gap> S2:=R3*R2*R3*R1*R2;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew
3
```

A.5.3 Structures of type $\mathsf{P}(S_1, S_2; v)$ with skew faces that are not polyhedra

Define $S_1 := R_0R_1R_2$.

```
S1:=R0*R1*R2;;
```

Define S_2 by $S_2 := R_2R_3R_1R_2R_3$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
S2:=R2*R3*R1*R2*R3;;
gap> G:=Group(S1,S2);;
```

```
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
2400
```

Define S_2 by $S_2 := (R_1R_2R_3)^2R_2$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=(R1*R2*R3)^2*R2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
2400
```

Define S_2 by $S_2 := R_2R_3R_1$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=R2*R3*R1;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
1440
```

Define S_2 by $S_2 := R_1R_2R_3R_1R_2$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=R1*R2*R3*R1*R2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
1440
```

Define $S_1 := R_3R_0R_1R_2R_3R_2R_1$.

```
gap> S1:=R3*R0*R1*R2*R3*R2*R1;;
```

Define S_2 as $S_2 := R_1(R_2R_3)^2R_1R_2R_3R_2$. Calculating the orbit size of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=R1*(R2*R3)^2*R1*R2*R3*R2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
1440
```

Defining S_2 as $S_2 := (R_1R_2R_3)^3$. Calculating the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=(R1*R2*R3)^3;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
1440
```

Defining S_2 as $S_2 := R_1(R_2R_3)^2R_1R_2$. Calculating the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=R1*(R2*R3)^2*R1*R2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
2400
```

Defining S_2 as $S_2 := (R_1R_2R_3)^2R_1$. Calculating the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=(R1*R2*R3)^2*R1;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
2400
```

Define $S_1 := R_0R_1R_2R_3R_1$.

```
gap> S1:=R0*R1*R2*R3*R1;;
```

Defining S_2 as $S_2 := R_3R_2R_3R_1R_2R_3R_2$. Calculating the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=R3*R2*R3*R1*R2*R3*R2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
2400
```

Defining S_2 as $S_2 := R_3R_2R_3R_1R_2$. Calculating the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=R3*R2*R3*R1*R2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
2400
```

Define S_2 as $S_2 := R_3R_2R_1$. Calculating the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=R3*R2*R1;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
1440
```

Defining S_2 as $S_2 := (R_3R_1R_2)^2R_3$. Calculating the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=(R3*R1*R2)^2*R3;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
```

```
gap> Index(G, H);
1440
```

Define$S_1 := R_3R_2R_3R_0R_1R_2R_3R_2R_1$.

```
gap> S1:=R3*R2*R3*R0*R1*R2*R3*R2*R1;;
```

Define S_2 as $S_2 := R_1(R_2R_3)^2R_1R_2$. Calculate the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=R1*(R2*R3)^2*R1*R2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
2400
```

Define S_2 by $S_2 := (R_1R_2R_3)^2R_1$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=(R1*R2*R3)^2*R1;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
2400
```

Define S_2 as $S_2 := R_1(R_2R_3)^2R_1R_2R_3R_2$. Calculating the orbit size of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=R1*(R2*R3)^2*R1*R2*R3*R2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
1440
```

Define S_2 as $S_2 := (R_1R_2R_3)^3$. Calculate the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=(R1*R2*R3)^3;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
1440
```

A.5.4 Candidates for S_2 in the 600-cell for helical S_1

$S_1 = R_0(R_1R_2R_3)^3$.

```
gap> S1:=R0*(R1*R2*R3)^3;; #Order 12 #Helical
gap> CandSigma2:=[];;
gap> for j in [1..Length(ElVF)] do
>       if (Order(ElVF[j])=1 or Order(ElVF[j])=2)=true then
>               continue;
>       else
```

```
>                   Add(CandSigma2,ElVF[j]);
>         fi;
> od;
gap> CandSigma2;;
gap> Length(CandSigma2);
88
gap> S2:=[];;
gap> for j in [1..Length(CandSigma2)] do
>         if Order(S1*CandSigma2[j])=2 then
>                   Add(S2,Factorization(P,CandSigma2[j]));
>         fi;
> od;
gap> S2;
[ x4*x3*x4*x2*x3*x4*x3*x2, x4*(x3*x4*x2)^3, (x4*x2*x3)^3*x2,
 x4*(x3*x4*x2)^2*x3*x2, (x4*x2*x3)^4 ]

gap> u:=v*S1;;

gap> S2:=R3*R2*R3*R1*R2*R3*R2*R1;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=R3*(R2*R3*R1)^3;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2

gap> S2:=(R3*R1*R2)^3*R1;;
gap> Order(S2);
3
gap> espacio(S2,u);  #Plane.
2

gap> S2:=R3*(R2*R3*R1)^2*R2*R1;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew.
3

gap> S2:=(R3*R1*R2)^4;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2
```

$\underline{S_1 = R_2R_3R_0R_1R_2R_3R_2R_1.}$

```
gap> S1:=R2*R3*R0*R1*R2*R3*R2*R1;;#Orden 15
gap> #We repeat the previous code to obtain the candidates for S2.
gap> S2;
[ (x2*x3*x4)^3*x3, x2*(x3*x4)^2*x2*x3, x2*(x3*x4)^2*x2*x3*x4,
```

```
 x2*(x3*x4)^2*x2, (x2*x3*x4)^2*x2*x3 ]

gap> u:=v*S1;;

gap> S2:=(R1*R2*R3)^3*R2;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane
2

gap> S2:=R1*(R2*R3)^2*R1*R2;;
gap> Order(S2);
6
gap> espacio(S2,u); #3 Skew.
3

gap> S2:=R1*(R2*R3)^2*R1*R2*R3;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=R1*(R2*R3)^2*R1;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2

gap> S2:=(R1*R2*R3)^2*R1*R2;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2
```

$\underline{S_1 = R_2R_3R_0R_1R_2R_3.}$

```
gap> S1:=R2*R3*R0*R1*R2*R3;; #Order 20.
gap> S2;
[ (x4*x2*x3)^2*x4*x3, x4*x2*x3*x2, x4*x3*x4*x2, (x4*x2*x3)^2,
 x4*x3*x4*x2*x3 ]

gap> u:=v*S1;;

gap> S2:=(R3*R1*R2)^2*R3*R2;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=R3*R1*R2*R1;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2
```

```
gap> S2:=R3*R2*R3*R1;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=(R3*R1*R2)^2;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2

gap> S2:=R3*R2*R3*R1*R2;;
gap> Order(S2);
6
gap> espacio(S2,u); #Skew.
3
```

$\underline{S_1 = R_0R_1R_2R_3.}$

```
gap> S1:=R0*R1*R2*R3;; #Order 30
gap> S2;
[ (x4*x2*x3)^2*x4*x3, x4*x2*x3*x2, x4*x3*x2, x4*x3*x4*x2, (x4*x2*x3)^2 ]

gap> u:=v*S1;;

gap> S2:=(R3*R1*R2)^2*R3*R2;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=R3*R1*R2*R1;;
gap> Order(S2);
5
gap> espacio(S2,u); #Plane.
2

gap> S2:=R3*R2*R1;;
gap> Order(S2);
10
gap> espacio(S2,u); #Skew.
3

gap> S2:=R3*R2*R3*R1;;
gap> Order(S2);
3
gap> espacio(S2,u); #Plane.
2

gap> S2:=(R3*R1*R2)^2;;
gap> Order(S2);
5
```

```
gap> espacio(S2,u); #Plane.
2
```

A.5.5 Structures of type $\mathsf{P}(S_1, S_2; v)$ with helical faces that are not polyhedra

Define $S_1 := R_0(R_1R_2R_3)^3$.

```
gap> S1:=R0*(R1*R2*R3)^3;;
```

Define S_2 as $S_2 := R_3(R_2R_3R_1)^3$. Calculating the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle$

```
gap> S2:=R3*(R2*R3*R1)^3;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
240
```

Define S_2 as $S_2 := (R_3R_1R_2)^4$. Calculate the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=(R3*R1*R2)^4;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
240
```

Define $S_1 := R_2R_3R_0R_1R_2R_3R_2R_1$.

```
gap> S1:=R2*R3*R0*R1*R2*R3*R2*R1;;
```

Define S_2 as $S_2 := (R_1R_2R_3)^3R_2$. Calculate the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=(R1*R2*R3)^3*R2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
480
```

Define S_2 as $S_2 := R_1(R_2R_3)^2R_1R_2R_3$. Calculating the orbit size of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=R1*(R2*R3)^2*R1*R2*R3;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
480
```

Define S_2 as $S_2 := R_1(R_2R_3)^2R_1$. Calculate the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=R1*(R2*R3)^2*R1;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
1440
```

Define S_2 as $S_2 := (R_1R_2R_3)^2R_1R_2$. Calculating the orbit size of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=(R1*R2*R3)^2*R1*R2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
1440
```

Define$S_1 := R_2R_3R_0R_1R_2R_3$.

```
gap> S1:=R2*R3*R0*R1*R2*R3;;
```

Define S_2 as $S_2 := (R_3R_1R_2)^2R_3R_2$. Calculate the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=(R3*R1*R2)^2*R3*R2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
240
```

Define S_2 as $S_2 := R_3R_2R_3R_1$. Calculate the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=R3*R2*R3*R1;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
240
```

Define S_2 by $S_2 := R_3R_1R_2R_1$. Compute the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and the index $[G, \langle S_2\rangle]$.

```
gap> S2:=R3*R1*R2*R1;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
240
```

Define S_2 as $S_2 := (R_3R_1R_2)^2$. Calculate the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=(R3*R1*R2)^2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
240
```

Define $S_1 := R_0R_1R_2R_3$.

```
gap> S1:=R0*R1*R2*R3;;
```

Define S_2 as $S_2 := (R_3R_1R_2)^2R_3R_2$. Calculate the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=(R3*R1*R2)^2*R3*R2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
480
```

Define S_2 as $S_2 := R_3R_2R_3R_1$. Calculate the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=R3*R2*R3*R1;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
480
```

Define S_2 as $S_2 := R_3R_1R_2R_1$. Calculate the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=R3*R1*R2*R1;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
1440
```

Define S_2 as $S_2 := (R_3R_1R_2)^2$. Calculate the size of the orbit of v under $G = \langle S_1, S_2\rangle$ and $[G, \langle S_2\rangle]$.

```
gap> S2:=(R3*R1*R2)^2;;
gap> G:=Group(S1,S2);;
gap> Size(Orbit(G, v, OnPoints));
120
gap> H:=Group(S2);;
gap> Index(G, H);
1440
```

A.5.6 A chiral polyhedron with helical faces, derived from the 600-cell

To prove that the polytope $\mathsf{P}(P_0P_1P_3P_2,\ P_2P_1; v)$ of type $\{12, 3\}$ that appears in [2] is isomorphic to $\mathsf{P}(S_1, R; v)$, we will first look for an element α in $[3, 3, 5]$, such that $\alpha(P_0P_1P_3P_2)\alpha^{-1} = S_1$ and $\alpha(P_2P_1)\alpha^{-1} = R$.

```
gap> S1:=R0*(R1*R2*R3)^3;;   #Order 12
gap> R:=R3*R2*R3*R1*R2*R3*R2*R1;; #Candidate to be S2.
```

To prove that $\mathsf{P}(S_1, Q; v)$ and $\mathsf{P}(S_1, R; v)$ are isometric polyhedra, we look for a $\alpha \in [3, 3, 5]$ such that $\alpha S_1\alpha^{-1} = S_1^{-1}$ and $\alpha Q\alpha^{-1} = R^{-1}$.

```
gap> Q:=(R3*R1*R2)^3*R1;;
gap> Alfa:=[];;
gap> for k in [1..Length(eGC)] do
>       if  eGC[k]*S1*eGC[k]^-1=S1^-1 and eGC[k]*Q*eGC[k]^-1=R^-1 then
>               Add(Alfa,Factorization(P,eGC[k]));
>       else
>               continue;
>       fi;
```

```
> od;
gap> Alfa;
[ x1*x2*(x3*x4)^2*(x2*x3*x4*x1*(x2*x3*x4)^2*x1)^3*x2*x3*x4*x1*x2*x3*x1
*x2*x1, x4*(x3*x4*x2)^3*x3*x4 ]
```

A.6 Chiral 4-polytopes

First, we define all functions that will be used in all cases.

Given symmetries S_1 and S_2, where P denotes the full group of one of the convex regular 4-polytopes, and a group G defined as $\langle R_2, R_3 \rangle$, the function `candS3(s1, s2, G, P)` returns the candidates for S_3.

```
gap> candS3 := function(s1, s2, G, P)
>      local eG, resultado, j;
>
>      eG := Elements(G);
>      resultado := [];
>
>      for j in [1..Length(eG)] do
>          if Order(s1 * s2 * eG[j]) = 2 and Order(s2 * eG[j]) = 2 and
Order(eG[j]) > 2 then
>              Add(resultado, Factorization(P, eG[j]));
>          fi;
>      od;
>
>      return resultado;
> end;
function( s1, s2, G, P ) ... end
```

The function StabE(v,e,G) returns the elements of the group $\langle S_1, S_2, S_3 \rangle$ that stabilize the base edge e, expressed as permutations.

```
gap> StabE:=function(v,e,G)
>        local eP, lStabE, i;
>
>        eP:=Elements(G);
>        lStabE:=[];
>        for i in [1..Length(eP)] do
>                if (Length(Cycle(eP[i],v))>1 and Cycle(eP[i],v)[2]=e and
Length(Cycle(eP[i],e))>1 and Cycle(eP[i],e)[2]=v) then
>                        Add(lStabE,eP[i]);
>                fi;
>                if (Length(Cycle(eP[i],v))=1 and Length(Cycle(eP[i],e))=1) then
>                        Add(lStabE,eP[i]);
>                fi;
>        od;
>        return lStabE;
> end;
function( v, e, G ) ... end
```

Given the base face f, the function face(f,G) returns all possible lists corresponding to the same base face f.

```
gap> face:=function(f,G)
>        local eG, fb, j, f1, f2;
>
>        eG := Elements(G);
>        fb:=[];
```

```
>        for j in [1..Length(eG)] do
>                f1:=OnTuples( f, eG[j] );
>                f2:=Reversed(f1);
>                Add(fb,f1);
>                Add(fb,f2);
>        od;
>        return fb;
> end;
function( f, G ) ... end
```

The function StabF(v,e,G) returns the elements of the group $\langle S_1, S_2, S_3 \rangle$ that stabilize the base edge f, expressed as permutations.

```
gap> StabF:=function(f,G,GS1)
>        local lStabF, i, eG;
>
>        eG := Elements(G);
>        lStabF:= [];
>        for i in [1..Length(eG)] do
>                if OnTuples( f, eG[i] ) in face(f,GS1) then
>                        Add(lStabF,eG[i]);
>                fi;
>        od;
>        return lStabF;
> end;
function( f, G, GS1 ) ... end
```

The cell function will give us the base cell c.

```
gap> celda:=function(f,G,U)
>        local transGU, j, cel;
>
>        transGU :=Elements(RightTransversal(G,U));
>        cel:=[];
>        for j in [1..Length(transGU)] do
>                Add(cel,OnTuples( f, transGU[j] ));
>        od;
>        return cel;
> end;
function( f, G, U ) ... end
```

Given a list fa, the function $nf(fa, k)$ returns a list shifted by k positions.

```
gap> nf := function(fa, k)
>      local ln, i, n;
>      n := Length(fa);
>      ln := [];
>      for i in [1..n] do
>          Add(ln, fa[(i + k - 1) mod n + 1]);
>      od;
>      return ln;
> end;
function( fa, k ) ... end
```

Given a face fa, the function $sf(fa)$ returns all possible lists corresponding to the same base face fa.

```
gap> sf:= function(fa)
>      local bc1, k, f1;
>
```

```
>      bc1 := [];
>      Add(bc1, fa);                # Original
>      Add(bc1, Reversed(fa));      # Reverso del original
>
>      for k in [1..(Length(fa)-1)] do
>          f1 := nf(fa, k);
>          Add(bc1, f1);             # Rotacion
>          Add(bc1, Reversed(f1));   # Reverso de la rotacion
>      od;
>
>      return bc1;
> end;
function( fa ) ... end
```

The function StabC(G,Celda,bc) returns the elements of the group $\langle S_1, S_2, S_3 \rangle$ that stabilize the base cell c, expressed as permutations.

```
gap> StabC:= function(G, Celda, bc)
>      local np, eG, lStabC, i, k, j, cont;
>
>      np := Length(Celda);
>      eG := Elements(G);
>      lStabC := [];
>
>      for i in [1..Length(eG)] do
>          cont := 0;
>          for k in [1..np] do
>              for j in [1..Length(bc)] do
>                  if OnTuples(Celda[k], eG[i]) in bc[j] then
>                      cont := cont + 1;
>                      break;  # Ya esta en bc, no necesita revisar mas
>                  fi;
>              od;
>          od;
>          if cont = np then
>              Add(lStabC, eG[i]);
>          fi;
>      od;
>
>      return lStabC;
> end;
function( G, Celda, bc ) ... end
```

The following functions are analogous to the previous ones; they are used for the 120-cell and 600-cell cases, since the groups are expressed in terms of matrices.

```
gap> Stabe:=function(v,e,G)
>        local eP, lStabE, i;
>
>        eP:=Elements(G);
>        lStabE:=[];
>        for i in [1..Length(eP)] do
>                if v*eP[i]=e and e*eP[i]=v then
>                        Add(lStabE,eP[i]);
>                fi;
```

```
>                 if v*eP[i]=v and e*eP[i]=e  then
>                         Add(lStabE,eP[i]);
>                 fi;
>         od;
>         return lStabE;
> end;
function( v, e, G ) ... end

gap> Stabf:=function(f,G,GS1)
>         local lStabF, i, j, eG, im;
>
>         eG := Elements(G);
>         lStabF:= [];
>         for i in [1..Length(eG)] do
>                 im:=[]; #Applying the symmetry eG[i] to f.
>                 for j in [1..Length(f)] do
>                         Add(im,f[j]* eG[i]);
>                 od;
>                 if im in sf(f) then
>                         Add(lStabF,eG[i]);
>                 fi;
>         od;
>         return lStabF;
> end;
function( f, G, GS1 ) ... end

gap> cel:=function(f,G,U)
>         local transGU, i, j, cell, im;
>
>         transGU :=Elements(RightTransversal(G,U));
>         cell:=[];
>         for i in [1..Length(transGU)] do
>                 im:=[]; #Applying the symmetry transGU[i] ) to f.
>                 for j in [1..Length(f)] do
>                         Add(im,f[j]* transGU[i]);
>                 od;
>                 Add(cell, im);
>         od;
>         return cell;
> end;
function( f, G, U ) ... end

gap> im:=function(Celda,S)
>         local m, l, ImC, im;
>
>         ImC:=[];
>         for m in [1..Length(Celda)] do
>                 im:=[]; #Image of each face.
>                 for l in [1..Length(Celda[1])] do
>                         Add(im,Celda[m][l]*S);
>                 od;
>                 Add(ImC,im);
>         od;
>         return ImC;
> end;
```

```
function( Celda, S ) ... end

gap> Stabc:= function(G, Celda, bc)
>      local np, eG, lStabC, i, j, k, cont;
>
>      np := Length(Celda);
>      eG := Elements(G);
>      lStabC := [];
>
>      for i in [1..Length(eG)] do
>          cont := 0;
>          for k in [1..np] do
>              for j in [1..Length(bc)] do
>                 if im(Celda,eG[i])[k] in bc[j] then
>                      cont := cont + 1;
>                      break;
>                  fi;
>              od;
>          od;
>          if cont = np then
>              Add(lStabC, eG[i]);
>          fi;
>      od;
>
>      return lStabC;
> end;
function( G, Celda, bc ) ... end
```

A.6.1 Searching for S_3 in the 5-cell

```
gap> r0:=(1,2);;
gap> r1:=(2,3);;
gap> r2:=(3,4);;
gap> r3:=(4,5);;
gap> P:=Group(r0,r1,r2,r3);
Group([ (1,2), (2,3), (3,4), (4,5) ])
gap> GS3:=Group(r2,r3);
Group([ (3,4), (4,5) ])
gap> S1:=r0*r1*r2;
(1,4,3,2)
gap> S2:=r2*r3*r1;
(2,3,5,4)
gap> S2p:=r1*r2*r3*r1*r2;
(2,5,3,4)
gap> candS3(S1,S2,GS3,P);
[ x4*x3 ]
gap> S3:=r3*r2;
(3,4,5)
```

The only symmetry that is a candidate for S_3 is $S_3 := R_3R_2$, where $S_1 = R_0R_1R_2$ and $S_2 := R_2R_3R_1$. Next, we define G by $G := \langle S_1, S_2, S_3 \rangle$.

```
gap> v:=1;
1
gap> G:=Group(S1,S2,S3);
```

```
Group([ (1,4,3,2), (2,3,5,4), (3,4,5) ])
gap> Stabv:=Elements(Stabilizer(G,v));
[ (), (4,5), (3,4), (3,4,5), (3,5,4), (3,5), (2,3), (2,3)(4,5), (2,3,4),
  (2,3,4,5), (2,3,5,4), (2,3,5), (2,4,3), (2,4,5,3), (2,4), (2,4,5),
  (2,4)(3,5), (2,4,3,5), (2,5,4,3), (2,5,3), (2,5,4), (2,5), (2,5,3,4),
  (2,5)(3,4) ]
gap> GS2S3:=Elements(Group(S2,S3));
[ (), (4,5), (3,4), (3,4,5), (3,5,4), (3,5), (2,3), (2,3)(4,5), (2,3,4),
  (2,3,4,5), (2,3,5,4), (2,3,5), (2,4,3), (2,4,5,3), (2,4), (2,4,5),
  (2,4)(3,5), (2,4,3,5), (2,5,4,3), (2,5,3), (2,5,4), (2,5), (2,5,3,4),
  (2,5)(3,4) ]
gap> e:=2;
2
gap> StabE(v,e,G);
[ (), (4,5), (3,4), (3,4,5), (3,5,4), (3,5), (1,2), (1,2)(4,5), (1,2)(3,4),
  (1,2)(3,4,5), (1,2)(3,5,4), (1,2)(3,5) ]
gap> Length(StabE(v,e,G));
12
gap> Ge:=Group(S3,S1*S2);
Group([ (3,4,5), (1,2)(4,5) ])
gap> Size(Ge);
6
gap> Elements(Ge);
[ (), (3,4,5), (3,5,4), (1,2)(4,5), (1,2)(3,4), (1,2)(3,5) ]
gap> StabE(v,e,G)[10];
(1,2)(3,4,5)
gap>  Factorization(G,StabE(v,e,G)[10]);
x2^-1*x3*x2*x1^-1
gap> Factorization(P,StabE(v,e,G)[10]);
x4*x3*x1
```

For $S2p$ defined as $S2p := R_1R_2R_3R_1R_2$.

```
gap> candS3(S1,S2p,GS3,P);
[ x3*x4 ]
gap> S3:=r2*r3;
(3,5,4)
gap> G:=Group(S1,S2p,S3);
Group([ (1,4,3,2), (2,5,3,4), (3,5,4) ])
gap> Stabv:=Elements(Stabilizer(G,v));
[ (), (4,5), (3,4), (3,4,5), (3,5,4), (3,5), (2,3), (2,3)(4,5), (2,3,4),
  (2,3,4,5), (2,3,5,4), (2,3,5), (2,4,3), (2,4,5,3), (2,4), (2,4,5),
  (2,4)(3,5), (2,4,3,5), (2,5,4,3), (2,5,3), (2,5,4), (2,5), (2,5,3,4),
  (2,5)(3,4) ]
gap> GS2S3:=Elements(Group(S2p,S3));
[ (), (4,5), (3,4), (3,4,5), (3,5,4), (3,5), (2,3), (2,3)(4,5), (2,3,4),
  (2,3,4,5), (2,3,5,4), (2,3,5), (2,4,3), (2,4,5,3), (2,4), (2,4,5),
  (2,4)(3,5), (2,4,3,5), (2,5,4,3), (2,5,3), (2,5,4), (2,5), (2,5,3,4),
  (2,5)(3,4) ]
gap> StabE(v,e,G);
[ (), (4,5), (3,4), (3,4,5), (3,5,4), (3,5), (1,2), (1,2)(4,5), (1,2)(3,4),
  (1,2)(3,4,5), (1,2)(3,5,4), (1,2)(3,5) ]
gap> Length(StabE(v,e,G));
12
```

```
gap> Ge:=Group(S3,S1*S2p);
Group([ (3,5,4), (1,2)(3,5) ])
gap> Size(Ge);
6
gap> Elements(Ge);
[ (), (3,4,5), (3,5,4), (1,2)(4,5), (1,2)(3,4), (1,2)(3,5) ]
gap> StabE(v,e,G)[10];
(1,2)(3,4,5)
gap> Factorization(G,StabE(v,e,G)[10]);
x2*x3^-1*x2*x1^-1
gap> Factorization(P,StabE(v,e,G)[10]);
x4*x3*x1
```

A.6.2 Searching for S_3 in the 5-simplex

```
gap> r0:=(1,2);
(1,2)
gap> r1:=(2,3);
(2,3)
gap> r2:=(3,4);
(3,4)
gap> r3:=(4,5);
(4,5)
gap> r4:=(5,6);
(5,6)
gap> P:=Group(r0,r1,r2,r3,r4);
Group([ (1,2), (2,3), (3,4), (4,5), (5,6) ])
gap> GS3:=Group(r2,r3,r4);  #Searching for $S_3$.
Group([ (3,4), (4,5), (5,6) ])
gap> #eGS3:=Elements(GS3);;
gap> #Length(eGS3);
gap> S1:=r0*r1*r2;
(1,4,3,2)
gap> S2:=r2*r3*r1;
(2,3,5,4)
gap> S2p:=r1*r2*r3*r1*r2;
(2,5,3,4)
gap> candS3(S1,S2,GS3,P);
[ x4*x3, x4*x5*x3 ]
```

We discard the first element of the list, as it was ruled out as an edge-based rotational symmetry within the 4-simplex skeleton.

```
gap> v:=1;;
gap> e:=2;;
gap> S3:=r3*r4*r2;
(3,4,6,5)
gap> G:=Group(S1,S2,S3);;
gap> Stabv:=Elements(Stabilizer(G,v));
[ (), (3,4,6,5), (3,5,6,4), (3,6)(4,5), (2,3)(4,6), (2,3,4,5,6), (2,3,5,4),
  (2,3,6,5), (2,4,5,3), (2,4)(5,6), (2,4,6,3,5), (2,4,3,6), (2,5,6,3),
  (2,5,4,6), (2,5)(3,4), (2,5,3,6,4), (2,6,5,4,3), (2,6,4,5), (2,6,3,4),
  (2,6)(3,5) ]
gap> GS2S3:=Elements(Group(S2,S3));;
gap> Stabv=GS2S3;
```

```
true
gap>
gap> StabE(v,e,G);
[ (), (3,4,6,5), (3,5,6,4), (3,6)(4,5), (1,2)(4,5), (1,2)(3,4)(5,6),
  (1,2)(3,5)(4,6), (1,2)(3,6) ]
  gap> Ge:=Elements(Group(S3,S1*S2));;
gap> StabE(v,e,G)=Ge;
true
gap> gap> f:=Cycle(S1,v);
[ 1, 4, 3, 2 ]
gap> GS1:=Group(S1);; #Group generated by S1.
gap> GS1:=Group(S1);; #Group generated by S1 .
gap> StabF(f,G,GS1);
[ (), (2,4)(5,6), (1,2)(3,4)(5,6), (1,2,3,4), (1,3)(5,6), (1,3)(2,4),
  (1,4,3,2), (1,4)(2,3)(5,6) ]
gap> Gf:=Elements(Group(S1,S2*S3));;
gap> StabF(f,G,GS1)=Gf;
true
gap>
gap> GS1S2:=Group(S1,S2);; #Group generated by S1 and S2 .
gap> Celda:=celda(f,GS1S2,GS1); #Faces of the base cell.
[ [ 1, 4, 3, 2 ], [ 2, 3, 5, 4 ], [ 3, 1, 2, 5 ], [ 4, 5, 1, 3 ],
  [ 5, 2, 4, 1 ] ]
gap> bc:=[ ];; #Base cell.
gap> for j in [1..Length(Celda)] do
>       Add(bc, sf(Celda[j]));
> od ;
gap> StabC(G,Celda,bc);
[ (), (2,3,5,4), (2,4,5,3), (2,5)(3,4), (1,2)(4,5), (1,2,3,4), (1,2,4,3,5),
  (1,2,5,3), (1,3,5,2), (1,3,4,5), (1,3)(2,4), (1,3,2,5,4), (1,4,3,2),
  (1,4)(3,5), (1,4,5,2,3), (1,4,2,5), (1,5,3,4,2), (1,5,4,3), (1,5)(2,3),
  (1,5,2,4) ]
gap> eGc:=Elements(GS1S2);;
gap> StabC(G,Celda,bc)=eGc;
true
```

Intersection property:

```
gap> Elements(Intersection(Group(S1),Group(S2)));
[ () ]
gap> Elements(Intersection(Group(S2),Group(S3)));
[ () ]
gap> Intersection(Group(S1,S2),Group(S2,S3))= Group(S2);
true
```

Orders of the groups:

```
gap> Size(G);
120
gap> Size(Group(S2,S3));
20
gap> Size(Group(S3,S1*S2));
8
gap> Size(Group(S1,S2*S3));
8
gap> Size(Group(S1,S2));
20
```

Therefore, the 4-polytope obtained has 6 vertices, 15 edges, 15 faces, and 6 cells.

```
gap> T2:=r1*r2*r3*r1*r2;
(2,5,3,4)
gap> candS3(S1,T2,GS3,P);
[ x3*x4, x3*x4*x5 ]
gap> T3:=r2*r3*r4;
```

Next, the groups $G := \langle S_1, S_2, S_3 \rangle$ and $H := \langle S_1, T_2, T_3 \rangle$ are defined, and it is shown that $G \cong H$.

```
gap> G:=Group(S1,S2,S3);
Group([ (1,4,3,2), (2,3,5,4), (3,4,6,5) ])
gap> H:=Group(S1,T2,T3);
Group([ (1,4,3,2), (2,5,3,4), (3,6,5,4) ])
gap> IsomorphismGroups(G, H);
[ (1,2)(3,4)(5,6), (1,2,3,6,4,5) ] -> [ (1,2)(3,4)(5,6), (1,2,3,5,4,6) ]
```

A.6.3 Searching for S_3 in the 8-cell

```
gap> r0:= (1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15);
(1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15)
gap> r1:= (2,4)(6,8)(10,12)(14,16);
(2,4)(6,8)(10,12)(14,16)
gap> r2:= (2,13)(3,16)(6,9)(7,12);
(2,13)(3,16)(6,9)(7,12)
gap> r3:= (5,13)(6,14)(7,15)(8,16);
(5,13)(6,14)(7,15)(8,16)
gap> P:=Group(r0,r1,r2,r3);
Group([ (1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15), (2,4)(6,8)(10,12)
  (14,16), (2,13)(3,16)(6,9)(7,12), (5,13)(6,14)(7,15)(8,16) ])
gap> eP:=Elements(P);;
gap> GS3:=Group(r2,r3);
Group([ (2,13)(3,16)(6,9)(7,12), (5,13)(6,14)(7,15)(8,16) ])
gap> eGS3:=Elements(GS3);;
gap> S1:=r0*r1*r2*r3;
(1,5,9,10,11,15,3,4)(2,8,13,6,12,14,7,16)
gap> S2:=r3*r2*r1*r3;
(2,5,4)(3,6,8)(9,16,14)(10,12,15)
gap> T2:=r3*r2*r1*r2;
(3,14,6)(4,13,5)(7,15,10)(8,16,9)
gap> v:=11;
11
gap> e:=10;
10
gap> candS3(S1,S2,GS3,P);
[ x3*x4 ]
gap> S3:=r2*r3;
(2,5,13)(3,8,16)(6,9,14)(7,12,15)

gap> G:=Group(S1,S2,S3);;
gap> Stabv:=Elements(Stabilizer(G,v));;
gap> GS2S3:=Elements(Group(S2,S3));;
gap> Stabv=GS2S3;
true
```

```
gap> StabE(v,e,G);;
gap> Ge:=Elements(Group(S3,S1*S2));;
gap> StabE(v,e,G)=Ge;
true

gap> f:=Cycle(S1,v);  #Base face.
[ 11, 15, 3, 4, 1, 5, 9, 10 ]
gap> GS1:=Group(S1);; #Group generated by S1.
gap> StabF(f,G,GS1);;
gap> Gf:=Elements(Group(S1,S2*S3));;
gap> StabF(f,G,GS1)=Gf;
true

gap> GS1S2:=Group(S1,S2);; #Group generated by S1 and S2.
gap> Celda:=celda(f,GS1S2,GS1);  #Faces of the base cell.
[ [ 11, 15, 3, 4, 1, 5, 9, 10 ], [ 11, 12, 8, 5, 1, 2, 14, 15 ],
  [ 11, 10, 6, 2, 1, 4, 16, 12 ], [ 12, 16, 13, 14, 2, 6, 7, 8 ],
  [ 9, 5, 8, 7, 3, 15, 14, 13 ], [ 9, 13, 16, 4, 3, 7, 6, 10 ] ]
gap> bc:=[];; #Base cell.
gap> for j in [1..Length(Celda)] do
>       Add(bc,sf(Celda[j]));
> od;
gap> eGc:=Elements(GS1S2);;
gap> StabC(G,Celda,bc)=eGc;
true
```

Intersection property:

```
gap> Elements(Intersection(Group(S1), Group(S2)));
[ () ]
gap> Elements(Intersection(Group(S2), Group(S3)));
[ () ]
gap> Intersection(Group(S1,S2), Group(S2,S3))=Group(S2);
true
```

Order of the groups:

```
gap> Size(G);
192
gap> Size(Group(S2,S3));
12
gap> Size(Group(S3,S1*S2));
6
gap> Size(Group(S1,S2*S3));
16
gap> Size(Group(S1,S2));
48
```

Therefore, the 4-polytope obtained has 16 vertices, 32 edges, 12 faces, and 4 cells.

```
gap> candS3(S1,S2,GS3,P);
[ x4*x3 ]
gap> T3:=r3*r2;
(2,13,5)(3,16,8)(6,14,9)(7,15,12)
```

Next, the groups $G := \langle S_1, S_2, S_3 \rangle$ and $H := \langle S_1, T_2, T_3 \rangle$ are defined, and it is shown that $G \cong H$.

```
gap> G:=Group(S1,S2,S3);;
gap> H:=Group(S1,T2,T3);;
gap> IsomorphismGroups(G, H);
[ (1,5,9,10,11,15,3,4)(2,8,13,6,12,14,7,16),
  (2,5,4)(3,6,8)(9,16,14)(10,12,15), (2,5,13)(3,8,16)(6,9,14)(7,12,15) ]
   ->
[ (1,15,6,13,11,5,16,7)(2,14,10,9,12,8,4,3),
  (1,14,3)(4,13,15)(5,10,7)(8,9,11), (1,14,6)(4,15,7)(5,13,10)(8,16,11) ]
```

A.6.4 Searching for S_3 in the 24-cell

```
gap> r0:=(1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15)(19,23);
(1,4)(2,3)(5,8)(6,7)(9,12)(10,11)(13,16)(14,15)(19,23)
gap> r1:=(2,24)(4,21)(5,19)(7,18)(8,14)(10,17)(12,20)(13,22)(15,23);
(2,24)(4,21)(5,19)(7,18)(8,14)(10,17)(12,20)(13,22)(15,23)
gap> r2:=(5,13)(6,14)(7,15)(8,16)(17,20)(21,24);
(5,13)(6,14)(7,15)(8,16)(17,20)(21,24)
gap> r3:=(2,13)(3,16)(6,9)(7,12)(18,20)(22,24);
(2,13)(3,16)(6,9)(7,12)(18,20)(22,24)
gap> P:=Group(r0,r1,r2,r3);
<permutation group with 4 generators>
gap> eP:=Elements(P);;
gap> GS3:=Group(r2,r3);
Group([ (5,13)(6,14)(7,15)(8,16)(17,20)(21,24), (2,13)(3,16)(6,9)(7,12)
  (18,20)(22,24) ])
gap> eGS3:=Elements(GS3);;
gap> Length(eGS3);
6
gap> S1:=r0*r1*r3*r2*r1*r2;
(1,2,6,10,11,12,16,4)(3,5,14,7,9,15,8,13)(17,20,23,22,21,24,19,18)
gap> S2:=r2*r1*r2*r3*r1*r2;
(2,4,5)(3,8,6)(9,14,16)(10,15,12)(17,18,23)(19,21,22)
gap>
gap> v:=11;
11
gap> e:=10;
10
gap> candS3(S1,S2,GS3,P);
[ x3*x4 ]
gap> S3:=r2*r3;
(2,13,5)(3,16,8)(6,14,9)(7,15,12)(17,18,20)(21,22,24)
gap> G:=Group(S1,S2,S3);;
gap> Stabv:=Elements(Stabilizer(G,v));;
gap> GS2S3:=Elements(Group(S2,S3));;
gap> Stabv=GS2S3;
true

gap> StabE(v,e,G);;
gap> Ge:=Elements(Group(S3,S1*S2));;
gap> StabE(v,e,G)=Ge;
true
```

```
gap> f:=Cycle(S1,v);
[ 11, 12, 16, 4, 1, 2, 6, 10 ]
gap> GS1:=Group(S1);; #Group generated by S1.
gap> Gf:=Elements(Group(S1,S2*S3));;
gap> StabF(f,G,GS1)=Gf;
true

gap> GS1S2:=Group(S1,S2); #Group generated by S1 and S2.
Group([ (1,2,6,10,11,12,16,4)(3,5,14,7,9,15,8,13)(17,20,23,22,21,24,19,18),
  (2,4,5)(3,8,6)(9,14,16)(10,15,12)(17,18,23)(19,21,22) ])
gap> Celda:=celda(f,GS1S2,GS1); #Faces of the base cell.
[ [ 11, 12, 16, 4, 1, 2, 6, 10 ], [ 11, 10, 9, 5, 1, 4, 3, 15 ],
  [ 11, 15, 14, 2, 1, 5, 8, 12 ], [ 12, 8, 7, 6, 2, 14, 13, 16 ],
  [ 9, 10, 6, 7, 3, 4, 16, 13 ], [ 9, 13, 14, 15, 3, 7, 8, 5 ] ]
gap> bc:=[];; #Base cell.
gap> for j in [1..Length(Celda)] do
>       Add(bc,sf(Celda[j]));
> od;
gap> eGc:=Elements(GS1S2);;
gap> StabC(G,Celda,bc)=eGc;
true
```

Intersection property:

```
gap> Elements(Intersection(Group(S1), Group(S2)));
[ () ]
gap> Elements(Intersection(Group(S2), Group(S3)));
[ () ]
gap> Intersection(Group(S1,S2), Group(S2,S3))=Group(S2);
true
```

Order of the groups:

```
gap> Size(G);
192
gap> Size(Group(S2,S3));
12
gap> Size(Group(S3,S1*S2));
6
gap> Size(Group(S1,S2*S3));
16
gap> Size(Group(S1,S2));
48
```

Therefore, the 4-polytope obtained has 16 vertices, 32 edges, 12 faces, and 4 cells.

```
gap> T2:=r1*r2*r3*r1;
(2,4,13)(3,16,14)(6,8,9)(7,12,10)(18,23,20)(19,24,22)
gap> candS3(S1,T2,GS3,P);
[ x4*x3 ]
gap> T3:=r3*r2;
(2,5,13)(3,8,16)(6,9,14)(7,12,15)(17,20,18)(21,24,22)
```

Next, the groups $G := \langle S_1, S_2, S_3 \rangle$ and $H := \langle S_1, T_2, T_3 \rangle$ are defined, and it is shown that $G \cong H$.

```
gap> G:=Group(S1,S2,S3);;
gap> H:=Group(S1,T2,T3);;
gap> IsomorphismGroups(G, H);
```

```
[ (1,2,6,10,11,12,16,4)(3,5,14,7,9,15,8,13)(17,20,23,22,21,24,19,18),
  (2,4,5)(3,8,6)(9,14,16)(10,15,12)(17,18,23)(19,21,22),
  (2,13,5)(3,16,8)(6,14,9)(7,15,12)(17,18,20)(21,22,24) ] ->
[ (1,7,16,2,11,13,6,12)(3,15,14,10,9,5,8,4)(17,22,24,23,21,18,20,19),
  (2,4,13)(3,16,14)(6,8,9)(7,12,10)(18,23,20)(19,24,22),
  (3,14,6)(4,13,5)(7,15,10)(8,16,9)(17,23,20)(19,24,21) ]
```

A.6.5 Searching for S_3 in the 120-cell

```
gap> rho := (1 + Sqrt(5)) / 2;; #Golden ratio..
gap> #Generating matrices of the 120-cell.
gap> r0 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, -1, 0],
>          [0, 0, 0, 1] ];;
gap> r1 := (1/2) * [ [2, 0, 0, 0],
>                    [0, rho, -1, rho - 1],
>                    [0, -1, 1 - rho, rho],
>                    [0, rho - 1, rho, 1] ];;
gap> r2 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, 1, 0],
>          [0, 0, 0, -1] ];;
gap> r3 := (1/2) * [ [rho, 1, 0, -1 + rho],
>                    [1, 1 - rho, 0, -rho],
>                    [0, 0, 2, 0],
>                    [-1 + rho, -rho, 0, 1] ];;
gap> v:=[1+rho,1,-2+rho,0];; #Base vertex.
gap> P:=Group(r0,r1,r2,r3);;
gap> eGC:=Elements(P);;
gap> e:=v*r0;;
gap> GS3:=Group(r2,r3);
<matrix group with 2 generators>
gap> eGS3:=Elements(GS3);;

gap> S1:=r0*r1*r2*r3;;
gap> S2:=r3*r2*r1*r3;;
gap> candS3(S1,S2,GS3,P);
[ x3*x4 ]
gap> S3:=r2*r3;;
gap> G:=Group(S1,S2,S3);;
gap> Stabv:=Elements(Stabilizer(G,v));;
gap> GS2S3:=Elements(Group(S2,S3));;
gap> Stabv=GS2S3;
true
gap>
gap> Stabe(v,e,G);;
gap> Ge:=Elements(Group(S3,S1*S2));;
gap> Stabe(v,e,G)=Ge;
true
gap>
gap> f:=[];;
gap> for i in [0..Order(S1)-1] do
>       Add(f,v*S1^i);
```

```
> od;
gap> GS1:=Group(S1);; #Group generated by S1.
gap> Stabf(f,G,GS1);;
gap> Gf:=Elements(Group(S1,S2*S3));;
gap> Stabf(f,G,GS1)=Gf;
true
gap> GS1S2:=Group(S1,S2);; #Group generated by S1 and S2.
gap> Celda:=cel(f,GS1S2,GS1);; #Faces of the base cell.
gap> bc:=[];; #Base cell.
gap> for j in [1..Length(Celda)] do
>       Add(bc,sf(Celda[j]));
> od;
gap> eGc:=Elements(GS1S2);;
gap> stabC:=Stabc(G,Celda,bc);;
gap> stabC=eGc;
true
```

Intersection property:

```
gap> Elements(Intersection(Group(S1), Group(S2)));
[ [ [ 1, 0, 0, 0 ], [ 0, 1, 0, 0 ], [ 0, 0, 1, 0 ], [ 0, 0, 0, 1 ] ] ]
gap> Elements(Intersection(Group(S2), Group(S3)));
[ [ [ 1, 0, 0, 0 ], [ 0, 1, 0, 0 ], [ 0, 0, 1, 0 ], [ 0, 0, 0, 1 ] ] ]
gap> Intersection(Group(S1,S2), Group(S2,S3))=Group(S2);
true
```

Orders from the following groups:

```
gap> Size(G);
7200
gap> Size(Group(S2,S3));
12
gap> Size(Group(S3,S1*S2));
6
gap> Size(Group(S1,S2*S3));
60
gap> Size(Group(S1,S2));
1440
```

Therefore, the 4-polytope obtained has 600 vertices, 1200 edges, 120 faces, and 5 cells.

```
gap> T2:=r3*r2*r1*r2;;
gap> candS3(S1,T2,GS3,P);
[ x4*x3 ]
gap> T3:=r3*r2;;
```

Next, the groups $G := \langle S_1, S_2, S_3 \rangle$ and $H := \langle S_1, T_2, T_3 \rangle$ are defined, and it is shown that $G \cong H$.

```
gap> G:=Group(S1,S2,S3);;
gap> H:=Group(S1,T2,T3);;
gap> LoadPackage("sonata");
gap> IsIsomorphicGroup(G,H);
true
```

A.6.6 Searching for S_3 in the 600-cell

```
gap> rho := (1 + Sqrt(5)) / 2;; #Golden ratio.
gap> #Generating matrices of the 600-cell.
gap> R0 := (1/2) * [ [rho, 1, 0, -1 + rho],
>                    [1, 1 - rho, 0, -rho],
>                    [0, 0, 2, 0],
>                    [-1 + rho, -rho, 0, 1] ];;
gap>
gap> R1 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, 1, 0],
>          [0, 0, 0, -1] ];;
gap>
gap> R2 := (1/2) * [ [2, 0, 0, 0],
>                    [0, rho, -1, rho - 1],
>                    [0, -1, 1 - rho, rho],
>                    [0, rho - 1, rho, 1] ];;
gap>
gap> R3 := [ [1, 0, 0, 0],
>          [0, 1, 0, 0],
>          [0, 0, -1, 0],
>          [0, 0, 0, 1] ];;
gap> v:=[1,0,0,0];; #Base vertex.
gap> P:=Group(R0,R1,R2,R3);;
gap> eGC:=Elements(P);;
gap> e:=v*R0;;
gap> GS3:=Group(R2,R3);
<matrix group with 2 generators>
gap> eGS3:=Elements(GS3);;
gap> S1:=R0*(R1*R2*R3)^3;;
gap> S2:=R3*R2*R3*R1*R2*R3*R2*R1;;

gap> candS3(S1,S2,GS3,P);
[ (x3*x4)^2 ]
gap> S3:=(R2*R3)^2;;

gap> G:=Group(S1,S2,S3);;
gap> Stabv:=Elements(Stabilizer(G,v));;
gap> GS2S3:=Elements(Group(S2,S3));;
gap> Stabv=GS2S3;

gap> Stabe(v,e,G);;
gap> Ge:=Elements(Group(S3,S1*S2));;
gap> Stabe(v,e,G)=Ge;
true
```

f will be the base face.

```
f:=[];;
for i in [0..Order(S1)-1] do
        Add(f,v*S1^i);
od;
GS1:=Group(S1);;
gap> Stabf(f,G,GS1);;
gap> Gf:=Elements(Group(S1,S2*S3));;
gap> Stabf(f,G,GS1)=Gf;
true
```

```
gap> GS1S2:=Group(S1,S2);; #Group generated by S1 and S2.
gap> Celda:=cel(f,GS1S2,GS1);; #Faces of the base cell.
gap> bc:=[];; #Base cell.
gap> for j in [1..Length(Celda)] do
>       Add(bc,sf(Celda[j]));
> od;
gap> eGc:=Elements(GS1S2);;
gap> stabC:=Stabc(G,Celda,bc);;
gap> stabC=eGc;
true
```

Intersection property:

```
gap> Elements(Intersection(Group(S1), Group(S2)));
[ [ [ 1, 0, 0, 0 ], [ 0, 1, 0, 0 ], [ 0, 0, 1, 0 ], [ 0, 0, 0, 1 ] ] ]
gap> Elements(Intersection(Group(S2), Group(S3)));
[ [ [ 1, 0, 0, 0 ], [ 0, 1, 0, 0 ], [ 0, 0, 1, 0 ], [ 0, 0, 0, 1 ] ] ]
gap> Intersection(Group(S1,S2), Group(S2,S3))=Group(S2);
true
```

Orders from the following groups:.

```
gap> Size(G);
7200
gap> Size(Group(S2,S3));
60
gap> Size(Group(S3,S1*S2));
10
gap> Size(Group(S1,S2*S3));
24
gap> Size(Group(S1,S2));
144
```

Therefore, the 4-polytope obtained has 120 vertices, 720 edges, 300 faces, and 50 cells.

```
gap> T2:=(R3*R1*R2)^3*R1;;
gap> candS3(S1,T2,GS3,P);
[ (x4*x3)^2 ]
gap> T3:=(R3*R2)^2;;
```

Next, the groups $G := \langle S_1, S_2, S_3 \rangle$ and $H := \langle S_1, T_2, T_3 \rangle$ are defined, and it is shown that $G \cong H$.

```
gap> G:=Group(S1,S2,S3);;
gap> H:=Group(S1,T2,T3);;

gap> LoadPackage("sonata");
gap> IsIsomorphicGroup(G,H);
true
```

Bibliography

[1] J. Bracho. Chiral polyhedra in the 3-sphere. https://www.matem.unam.mx/~roli/PetrieCoxeter.html, 2021.

[2] J. Bracho, I. Hubard, and D. Pellicer. Chiral polyhedra in 3-dimensional geometries and from a petrie-coxeter construction. *Discret. Comput. Geom.*, 66:1025–1052, 2021.

[3] M. Conder. Regular orientable maps of genus 2 to 301. https://www.math.auckland.ac.nz/~conder/RegularOrientableMaps301.txt, 2012.

[4] The GAP Group. *GAP – Groups, Algorithms, and Programming, Version 4.8.10*, 2018.